

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY VIZIANAGARAM

VIZIANAGARAM – 535003
ANDHRA PRADESH

SMART AGRICULTURE MONITORING APP

By

KANAKALA ASHOK

(Regd. No. 24VV1A1228)

Under the supervision of

MRS. MADHUMITA CHANDA

Mrs. GEETHA MADHURI YADAV

Department of Information Technology

JNTU-GV, Vizianagaram

Academic Year: 2026–2027



JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY

VIZIANAGARAM

VIZIANAGARAM – 535003

ANDHRA PRADESH

Regd.No:24VV1A1228

This is to certify that this is a bonafide record of practical work done by KANAKALA ASHOK of III B.Tech I Semester Class in USER INTERFACE DESIGN USING FLUTTER Lab during the year 2026-27.

No. of Tasks Completed and Certified:

Lecture In-Charge **Head of The Department**

Date:

DEPARTMENT OF INFORMATION TECHNOLOGY
JNTU-GURAJADA VIZIANAGARAM
COLLEGE OF ENGINEERING VIZIANAGARAM (A)
VIZIANAGARAM

Website:www.jntugvcev.edu.in

Subject Name: USER INTERFACE DESIGN USING FLUTTER Subject Code: R233112ES01

Academic Year: 2026 **Regulation: R23**

COURSE OUTCOMES

NBA Subject Code	Course Outcomes	
R233112ES01	CO1	Learns to Implement Flutter Widgets and Layouts.
	CO2	Understands Responsive UI Design and with Navigation in Flutter.
	CO3	Knowledge on Widgets and customize widgets for specific UI elements, Themes
	CO4	Understand to include animation apart from fetching data.

CO-PO Mapping

Mapping of Course Outcomes (COs) with Program Outcomes (POs)

Course Outcomes		Program Outcomes (POs)														
		PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PO 12	PSO 1	PSO 2	PSO 3
R233112ES01	CO1	3	2	3	2	3	1	1	1	2	2	1	2	3	3	2
	CO2	3	3	3	2	3	1	2	2	2	3	2	2	3	3	2
	CO3	3	2	3	2	3	2	1	1	2	2	1	2	3	2	2
	CO4	3	3	3	3	3	2	1	2	3	3	2	3	3	3	3

Enter correlation levels 1,2 and 3 as defined below:

1: Slight (Low) 2: Moderate (Medium) 3: Substantial (High) If there is no correlation, put "-"

Signature of the Course Instructor

Index

S. No	Date	Contents	Page No	Marks	Signature
1	01-07-2026	a) Introduction b) Project Structure	1 – 4		
2	08-07-2026	a) Database Setup and Dependencies b) Installation of Flutter (with PATH Setup)	5 – 10		
3	22-07-2026	a) Installation of Dart SDK in Windows b) Installing Visual Studio Code	11 – 12		
4	05-08-2026	a) Creating a New Flutter Project b) Dart Basic Programming	13 – 17		
5	12-08-2026	a) Case Study on Smart Agriculture Monitoring App b) Exploring Flutter Widgets	18 – 22		
6	19-08-2026	a) Responsive UI Design b) Navigation Between Screens	23 – 27		
7	02-09-2026	a) Stateful and Stateless Widgets b) Custom Widgets and Styling	28 – 34		
8	16-09-2026	a) Forms and Validation b) Animations	35 – 40		
9	30-09-2026	a) REST API Integration b) Testing and Debugging	41 – 46		
10	07-10-2026	a) Workflow of Screens b) Conclusion	47 – 54		

1. Introduction

Flutter is an open-source software development framework created by Google. It allows developers to build high-quality, natively compiled applications for mobile, web, and desktop from a single codebase. Flutter applications are written in the Dart programming language and are composed entirely of widgets, which describe how the user interface should look and behave.

Advantages of Flutter

- Cross-platform development with a single codebase.
- Hot reload for instant updates during development.
- Rich set of customisable widgets for every platform.
- High performance due to AOT compilation to native machine code.
- Backed by Google and a large, active community.

Dart Language

Dart is the programming language used by Flutter. It is object-oriented, class-based and supports both JIT and AOT compilation. Its syntax is similar to Java and C++, and it adds null safety, sound typing, collection literals and asynchronous programming with Future, async and await.

Advantages of Dart

- Easy-to-learn syntax that is familiar to Java and C++ programmers.
- Null safety, which prevents many runtime errors.
- Strong typing with type inference through var, final and const.
- Built-in support for asynchronous programming using async and await.

Key Features of Flutter

- **Cross-Platform Development:** One codebase for Android, iOS and web.
- **Hot Reload:** Code changes appear instantly without restarting the application.
- **Widget-Based Framework:** Every layout, button and text is a widget.
- **High Performance:** Apps are compiled to native ARM code for smooth animation.
- **Custom UI Design:** Material and Cupertino widgets give a native look on each platform.

About This Project

The **Smart Agriculture Monitoring App** is a Flutter application that keeps all important farm information in one place. It supports login, farm and crop records, soil moisture and water readings, live weather from an online API, irrigation advice in litres, automatic alerts, an offline agricultural assistant and a crop scanner with scan history. All records are stored in a local SQLite database, so the app keeps working without an internet connection.

This document explains the Flutter and Dart fundamentals used in the project, the module design, the source code and the screen workflow of the application.

2. Project Structure

The **Smart Agriculture Monitoring App** is a Flutter-based mobile application designed to create a simple digital environment for farm and crop monitoring. A farmer can register an account, maintain farm information, add crops, record soil and water readings, view current weather, calculate irrigation needs, receive alerts, and ask the built-in agricultural assistant for guidance. The app demonstrates real-world Flutter practices including navigation, widget composition, form validation, state management, custom painting and integration with a REST API and a local SQLite database.

Below is the directory structure of the project:

```
smart_agriculture_app/
├── lib/
│   ├── main.dart
│   ├── database/
│   │   ├── database_helper.dart
│   │   ├── db_factory.dart
│   │   ├── db_factory_io.dart
│   │   ├── db_factory_web.dart
│   │   └── db_factory_stub.dart
│   ├── models/
│   │   ├── user.dart
│   │   ├── farm.dart
│   │   ├── crop.dart
│   │   ├── alert.dart
│   │   ├── soil_reading.dart
│   │   ├── crop_scan.dart
│   │   ├── chat_message.dart
│   │   └── weather_data.dart
│   ├── services/
│   │   ├── session_service.dart
│   │   ├── weather_service.dart
│   │   ├── location_service.dart
│   │   ├── alert_service.dart
│   │   ├── soil_water_service.dart
│   │   └── ai_service.dart
│   ├── screens/
│   │   ├── main_navigation_screen.dart
│   │   ├── placeholder_screen.dart
│   │   ├── splash/splash_screen.dart
│   │   ├── welcome/welcome_screen.dart
│   │   ├── auth/login_screen.dart
│   │   ├── auth/register_screen.dart
│   │   ├── dashboard/dashboard_screen.dart
│   │   ├── farm/my_farms_screen.dart
│   │   ├── farm/add_edit_farm_screen.dart
│   │   ├── farm/crop_monitoring_screen.dart
```

```
│   │   ├── farm/add_edit_crop_screen.dart
│   │   ├── moisture/soil_moisture_screen.dart
│   │   ├── ai/ai_chat_screen.dart
│   │   ├── ai/ai_crop_scanner_screen.dart
│   │   ├── ai/ai_crop_advice_screen.dart
│   │   ├── alerts/alerts_screen.dart
│   │   ├── weather/weather_screen.dart
│   │   ├── profile/profile_screen.dart
│   │   └── debug/database_viewer_screen.dart
│   └── widgets/
│       ├── animated_background.dart
│       ├── auth_background.dart
│       ├── dashboard_background.dart
│       ├── farm_background.dart
│       ├── natural_landscape.dart
│       ├── alert_style.dart
│       ├── date_formats.dart
│       └── empty_state.
├── android/app/src/main/AndroidManifest.xml
├── ios/Runner/Info.plist
├── web/index.html, manifest.json, sqlite3.wasm, sqflite_sw.js
├── analysis_options.yaml
├── pubspec.yaml / pubspec.lock
├── README.md
└── test/
    ├── widget_test.dart
    ├── soil_water_service_test.dart
    └── layout_test.dart
```

Description of Project Modules

- **lib/** - The core directory of the Flutter app containing all Dart source files, organised into subfolders for modularity and clarity.
- **main.dart** - The entry point of the application. It initialises the database factory, starts the app, and defines the Material 3 theme, the colour palette (AppColors) and the text styles used by every screen.
- **database/** - Contains the SQLite layer. database_helper.dart creates the tables and exposes all insert, update, delete and query methods. The db_factory files select the correct database factory for Android, iOS, Windows, Linux or Web.
- **models/** - Defines the data structures of the app: User, Farm, Crop, SoilReading, Alert, CropScan, ChatMessage and WeatherData. Every model provides fromMap() and toMap() methods so that objects can be stored in and read from SQLite.
- **screens/** - Contains all the user interface screens, grouped by feature: splash, welcome, auth, dashboard, farm, moisture, weather, ai, alerts, profile and debug.
- **services/** - Contains the business logic: SessionService keeps the logged-in user, WeatherService calls the Open-Meteo REST API, LocationService reads the GPS position,

SoilWaterService calculates the daily water requirement, AlertService generates smart alerts and AIService answers agricultural questions offline.

- **widgets/** - Contains reusable presentation widgets such as EmptyState, FarmBackground, AnimatedBackground, DashboardBackground, AlertStyle and DateFormats helpers.
- **assets/** - Stores static resources. The application icon is kept in assets/images/app_icon.png and is registered in pubspec.yaml.
- **test/** - Contains the unit and widget tests for the calculator service, the layouts and the startup screen.

Packages Used

Package	Version	Purpose in the project
http	^1.2.2	Calls the Open-Meteo weather REST API
geolocator	^13.0.2	Reads the current GPS position of the device
permission_handler	^11.3.1	Requests location permission at runtime
image_picker	^1.1.2	Captures or selects a crop photograph
speech_to_text	^7.0.0	Voice input for the agricultural assistant
flutter_tts	^4.2.0	Reads assistant answers aloud
sqflite	^2.4.1	SQLite database on Android and iOS
sqflite_common_ffi	^2.3.0	SQLite on Windows, Linux and tests
sqflite_common_ffi_web	^1.1.0	SQLite when the app runs in a browser
shared_preferences	^2.5.3	Saves the logged-in user id
path / path_provider	^1.9.0 / ^2.1.5	Builds the database file path

3. Database Setup and Dependencies

The Smart Agriculture Monitoring App stores its data in **SQLite**, a lightweight relational database that is embedded in the application. No separate database server is required: the database is a single file created on the device the first time the application starts. Flutter accesses SQLite through the sqflite package. On Windows, Linux and in the automated tests the sqflite_common_ffi package provides the same API, and on the web the sqflite_common_ffi_web package is used. The package shared_preferences stores the id of the logged-in user so that the session survives an application restart.

The steps to add and configure the database for a Flutter project are as follows:

1. Add the packages to pubspec.yaml:

```
dependencies:
  flutter:
    sdk: flutter


  sqflite: ^2.4.1
  sqflite_common_ffi: ^2.3.0
  sqflite_common_ffi_web: ^1.1.0
  path: ^1.9.0
  path_provider: ^2.1.5
  shared_preferences: ^2.5.3
```

Then run the following command in the project folder to download and link the packages:

```
flutter pub get
```

2. Select the platform database factory:

sqflite uses a platform channel on Android and iOS, but on desktop and web it needs an FFI factory. A conditional import keeps a single entry point for all platforms:

db_factory.dart

```
export 'db_factory_stub.dart'
    if (dart.library.io) 'db_factory_io.dart'
    if (dart.library.js_interop) 'db_factory_web.dart';
```

db_factory_io.dart

```
import 'dart:io';


import 'package:sqflite/sqflite.dart';
import 'package:sqflite_common_ffi/sqflite_ffi.dart';


Future<void> initializeDatabaseFactory() async {
  if (Platform.isWindows || Platform.isLinux) {
  sqfliteFfiInit();
    databaseFactory = databaseFactoryFfi;
  }
}
```

3. Create the DatabaseHelper singleton:

A single DatabaseHelper instance opens the database once and reuses the connection. The tables are created in onCreate and later schema changes are applied in onUpgrade, while foreign key enforcement is switched on in onConfigure:

database_helper.dart

```
class DatabaseHelper {
  DatabaseHelper._();

  static final DatabaseHelper instance = DatabaseHelper._();

  static const String _dbName = 'smart_agriculture.db';
  static const int _dbVersion = 2;
  static Database? _database;

  Future<Database> get database async {
    _database ??= await _initDatabase();
    return _database!;
  }

  Future<Database> _initDatabase() async {
    await initializeDatabaseFactory();
    final dbPath = await getDatabasesPath();
    final path = join(dbPath, _dbName);
    return openDatabase(
      path,
      version: _dbVersion,
      onConfigure: (db) => db.execute('PRAGMA foreign_keys = ON'),
      onCreate: _onCreate,
      onUpgrade: _onUpgrade,
    );
  }
```

4. Create the tables:

Each feature of the application has its own table. The users table is created as shown below; the farms, crops, alerts, crop_scans, chat_history and soil_readings tables follow the same pattern.

```
Future<void> _onCreate(Database db, int version) async {
  await db.execute('''
    CREATE TABLE users(
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      name TEXT NOT NULL,
      email TEXT NOT NULL UNIQUE,
      phone TEXT,
      password TEXT NOT NULL
    )
  ''');

  await db.execute('''
    CREATE TABLE farms(
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      userId INTEGER NOT NULL,
      farmName TEXT NOT NULL,
```

```
    location TEXT,
    farmSize TEXT,
    soilType TEXT,
    mainCrop TEXT,
    description TEXT
  )
 ''');

 await _createSoilReadings(db);
}
```

Table	Important columns	Used for
users	id, name, email, phone, password	Registration and login
farms	id, userId, farmName, location, farmSize, soilType, mainCrop	Farm records of each user
crops	id, farmId, cropName, plantingDate, growthStage, status	Crop monitoring
soil_readings	farmId, moisturePercent, waterLevelPercent, waterUsedLitres, waterRequiredLitres, createdAt	Soil and water history
alerts	userId, title, message, type, isRead, createdAt	Smart alerts
crop_scans	userId, cropName, imagePath, condition, problem, care	Crop scan history
chat_history	userId, question, answer, createdAt	Assistant conversation

5. Verify the installation:

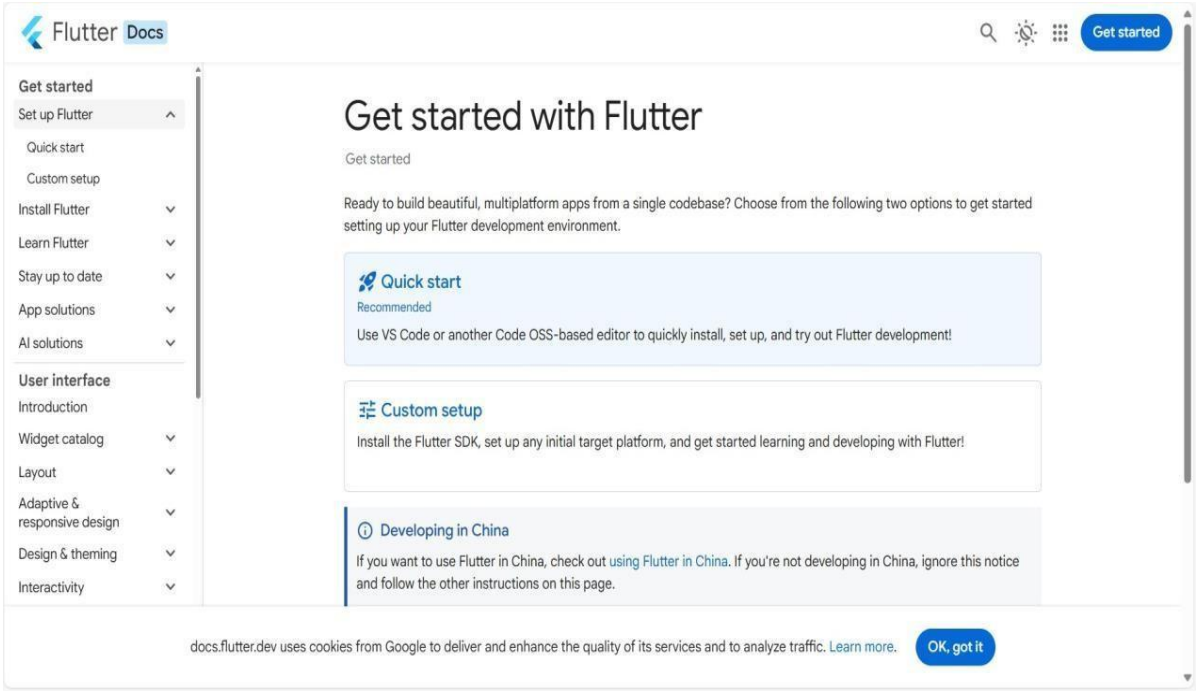
- Run the app on an emulator, a device or as a Windows application.
- The console prints "DIAG factory ok" after the database factory is initialised.
- Open the debug database viewer from the profile menu to confirm that the tables exist and that records are being stored.

Because the database is created and opened through DatabaseHelper, the screens never write SQL directly. They call typed methods such as addFarm(), getCropsForUser() or addSoilReading(), which keeps the SQL in one place and makes the code easier to test.

4. Installation of Flutter (with PATH Setup)

Flutter SDK Installation

1. Install the stable Flutter SDK and add its bin directory to PATH.
2. Open a new terminal and verify the installation with flutter --version.
3. Run flutter doctor to identify missing development components.
4. Install the Android SDK/platform tools and accept the required Android licenses.
5. Connect a physical Android device or configure an Android emulator.



Installation of Flutter

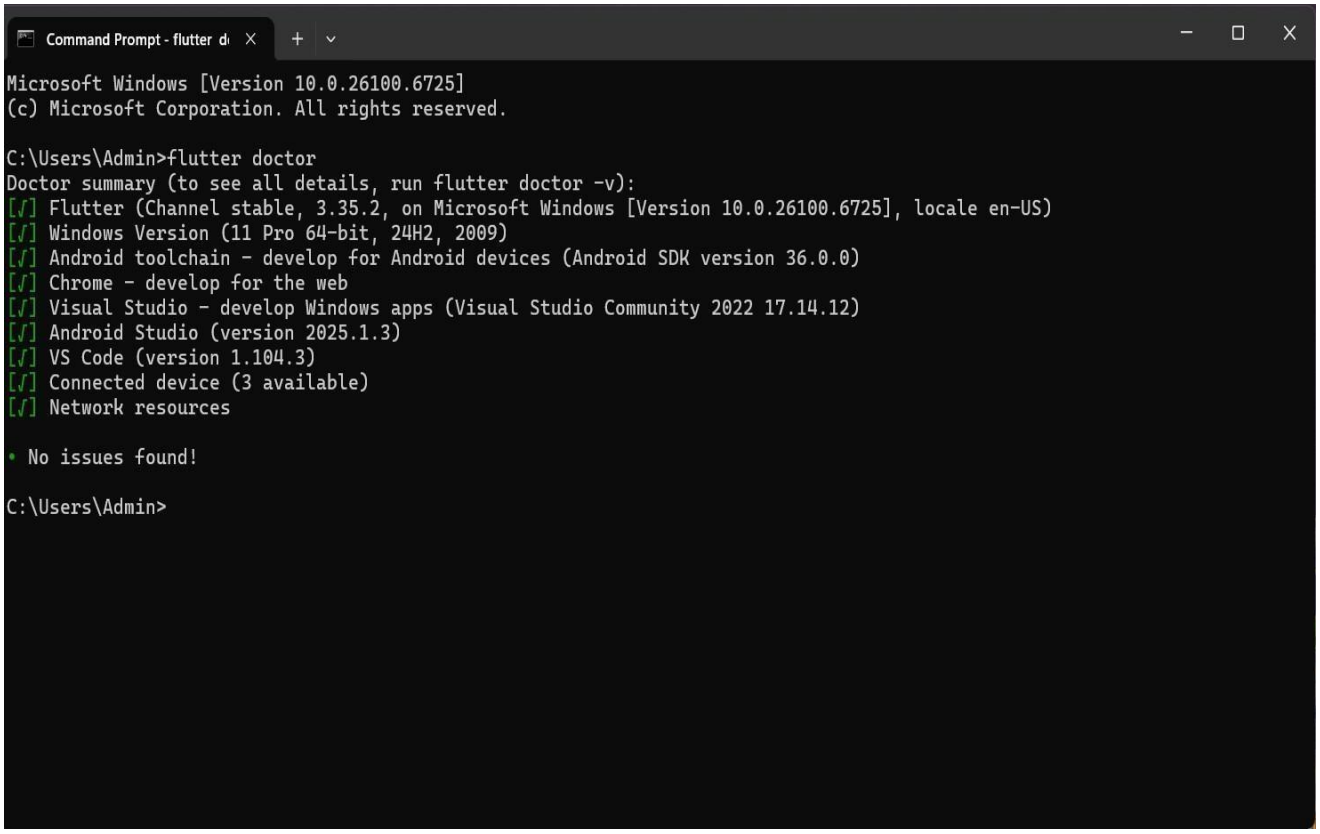
Project Environment Used

Component	Project Environment
Flutter	3.44.9 stable
Dart	3.12.2
UI	Material 3
Editor	Visual Studio Code
Database	SQLite with sqflite
Android SDK	36.0.0
Java/Kotlin	Java 17 / JVM 17

Verify the Environment

```
flutter --version
dart --version
flutter doctor
flutter devices
flutter pub get
```

The development workflow uses VS Code for source editing and the Flutter command line for package installation, application execution, analysis and testing.



Verifying the Environment

Installation of Android Studio

Step 1: Download and Install

1. Go to: https://developer.android.com/studio.
2. Download Android Studio.
3. Install with default settings.

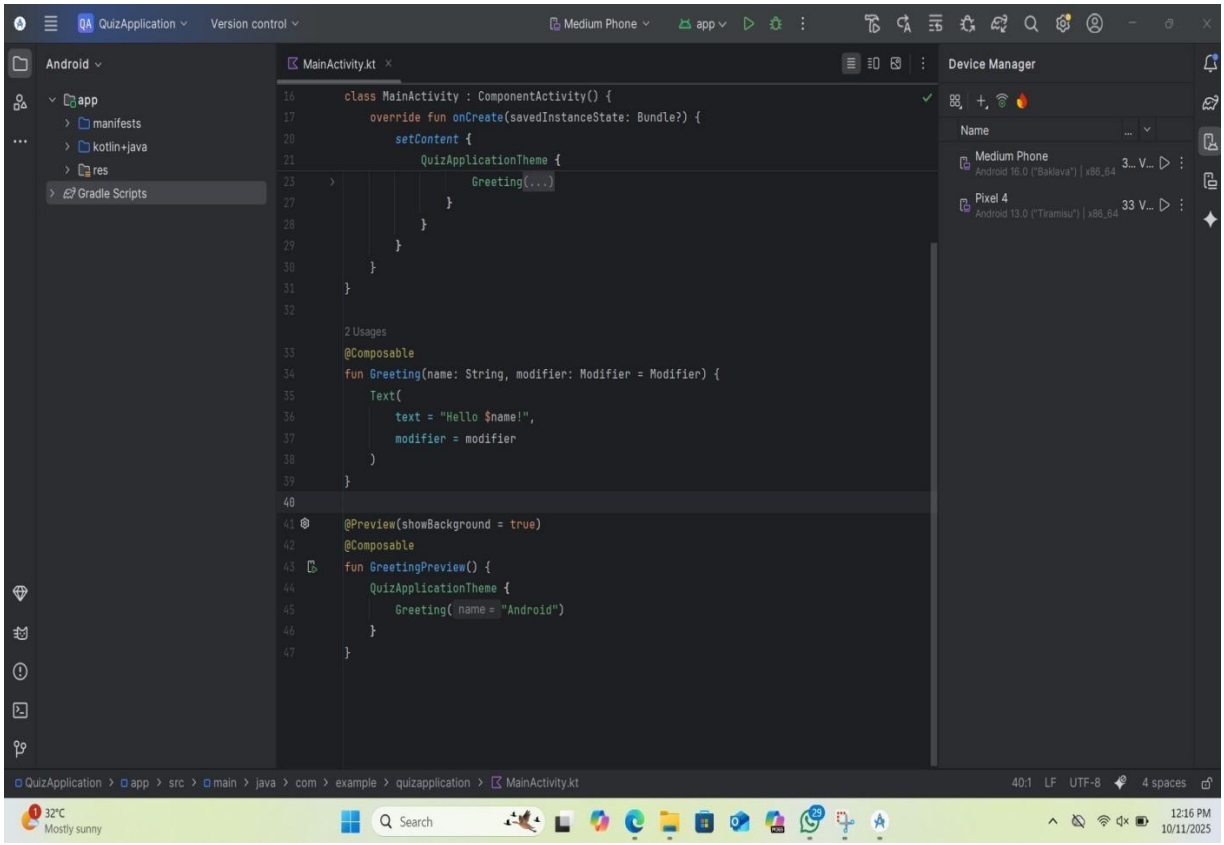
Step 2: Install Plugins

1. Open Android Studio.
2. Go to File → Settings → Plugins.

3. Install Flutter (Dart installs automatically).

Step 3: Configure SDK Path

1. Open SDK Manager.
2. Verify Android SDK location.
3. Install required SDK platforms & tools.



Android Studio Interface Showing Project Workspace

Emulator Setup and Running

1. Open Android Studio.
2. Go to Tools → AVD Manager.
3. Click Create Virtual Device.
4. Choose a device model (e.g., Pixel 6).
5. Select a system image (Android 13 or latest).
6. Finish and launch emulator.

5. Installation of Dart SDK in Windows

Dart is a programming language developed by Google, mainly used for building mobile, desktop, web and server applications. The Flutter SDK already bundles a compatible Dart SDK, but it can also be installed separately for running Dart programs from the command line. The following steps describe how to install the Dart SDK on a Windows system.

1. Download the Dart SDK:

- Go to the official Dart SDK website: https://dart.dev/get-dart.
- Under the Windows section, choose the appropriate version (64-bit recommended) and download the ZIP file.

2. Extract the Dart SDK:

- After downloading, extract the ZIP file to a preferred location, for example C:\dartsdk.

3. Set the environment variable:

- Right-click This PC, then Properties, then Advanced system settings.
- Click Environment Variables.
- Under System variables, select Path and click Edit.
- Click New and add the bin folder inside the Dart SDK, for example C:\dartsdk\bin.

4. Verify the installation:

- Open the Command Prompt.
- Run the following command to verify that Dart is installed successfully:

```
dart --version
```

5. Output: The terminal should display the installed Dart version, for example:

```
Dart SDK version: 3.12.2 (stable) (Tue Oct 7 10:12:31 2026 +0000)
on "windows_x64"
```

Once the Dart SDK is installed and configured, it can be used to compile and run Dart programs or to build Flutter applications. The project declares the constraint sdk: ^3.12.2 in pubspec.yaml, so the installed Dart version must satisfy this requirement.

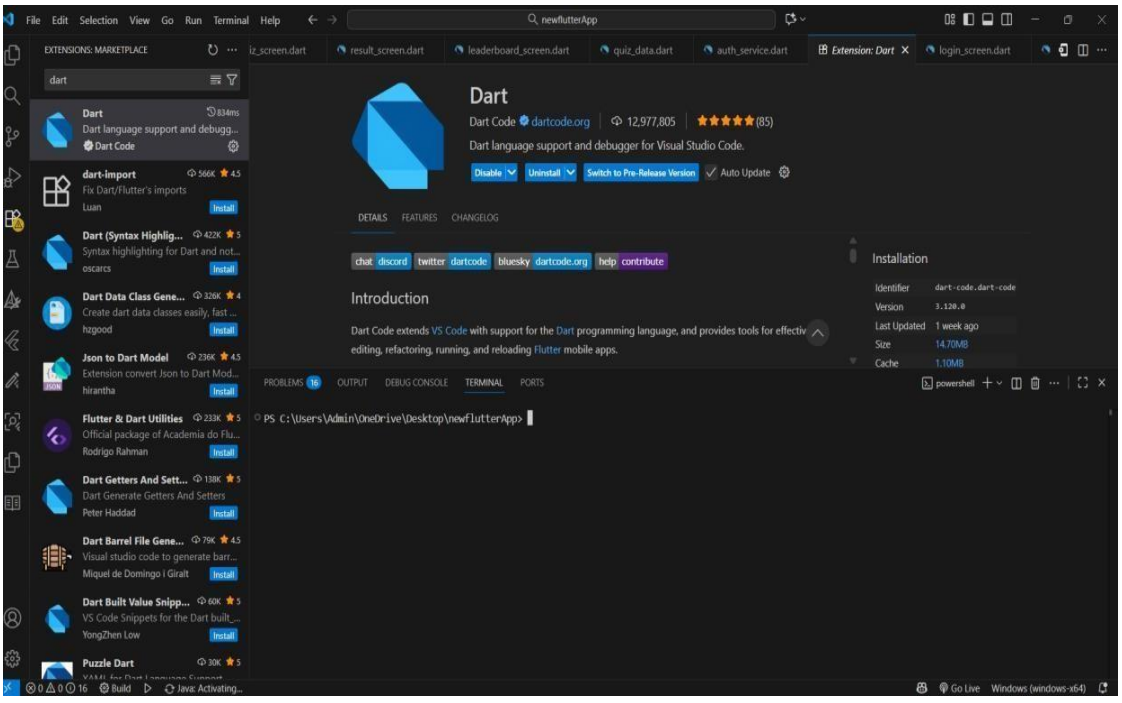
6. Installing Visual Studio Code

Visual Studio Code (VS Code) is a lightweight yet powerful open-source code editor developed by Microsoft. It is widely used for Flutter and Dart development because of its excellent support for extensions, debugging, IntelliSense, and integration with the Flutter SDK. By installing VS Code, developers can write, test and debug Flutter applications efficiently across multiple platforms.

Steps to Install VS Code and Set Up for Flutter Development

- **Download VS Code:** Visit https://code.visualstudio.com/ and download the version suitable for the operating system (Windows, macOS or Linux).
- **Install VS Code:** Run the downloaded installer and complete the installation using the default settings. This ensures that all the necessary components, including the PATH configuration, are set up.
- **Install extensions:** After launching VS Code, open the Extensions view with Ctrl + Shift + X and install the Flutter and Dart extensions.
- **Create a new Flutter project:** Press Ctrl + Shift + P, search for "Flutter: New Project", select Application, and give the project the name smart_agriculture_app.
- **Open the project in VS Code:** VS Code opens the folder automatically. The folders lib/, android/, ios/, web/ and test/ are generated by Flutter.

The Flutter extension adds the following useful commands to the editor: Flutter: Run, Flutter: Hot Reload, Flutter: Hot Restart, Flutter: Select Device and Dart: Open DevTools. The Debug Console shows the output of print() statements and any exceptions raised by the application.



Installing the Dart and Flutter extensions in Visual Studio Code

7. Creating a New Flutter Project

Flutter is a framework developed by Google for building cross-platform applications using the Dart programming language. Before creating a new Flutter project, ensure that the Flutter SDK is installed and added to the system PATH.

Steps to Create a New Flutter Project

1. Open the Command Prompt or a terminal.

2. Navigate to the folder in which the project should be created:

```
cd C:\Users\<your-username>\Dart
```

3. Create a new Flutter project:

```
flutter create smart_agriculture_app
```

4. Navigate into the project directory:

```
cd smart_agriculture_app
```

5. Open the project in Visual Studio Code:

```
code .
```

6. Run the Flutter application on the selected device:

```
flutter devices
flutter run
```

7. Verify the successful installation: once the app runs, the default counter application appears on the emulator or connected device. The counter code in lib/main.dart is then replaced with the source files of the Smart Agriculture Monitoring App.

Project Structure

After running the above commands, Flutter automatically generates the following project structure:

- **lib/** - Contains the main Dart source code.
- **android/, ios/, web/, windows/, linux/, macos/** - Platform-specific code generated by Flutter.
- **pubspec.yaml** - Configuration file for dependencies and assets.
- **test/** - Contains test files for unit and widget testing.
- **assets/** - Directory to store images, fonts and other static resources; it is added manually when needed.

8. Dart Basic Programming

Dart is a modern, object-oriented programming language developed by Google. It is optimised for building client applications, particularly mobile apps using the Flutter framework. Dart provides features such as strong typing, asynchronous programming and a rich standard library, which make it both easy to learn and powerful for application development. The following sections explain the fundamental programming concepts used in this project.

Variables

Variables are containers that hold data values. In Dart every variable has a name, a type and a value. Dart is a statically typed language, so the data type of a variable is known at compile time, but it also supports type inference through the var keyword.

- **String:** Stores textual data.
- **int:** Stores whole numbers.
- **double:** Stores decimal numbers.
- **bool:** Stores Boolean values, true or false.
- **var:** Automatically infers the type from the assigned value.
- **const:** Declares a compile-time constant whose value is fixed at compile time.
- **final:** Declares a variable whose value can be set only once, but is determined at runtime.

Code:

```
void main() {
  String farmer = 'Ashok';
  int farms = 3;
  double moisture = 42.5;
  bool needsWater = true;


  print('Farmer: $farmer');
  print('Farms: $farms');
  print('Moisture: $moisture%');
  print('Needs water: $needsWater');
}
```

Output:

```
Farmer: Ashok
Farms: 3
Moisture: 42.5%
Needs water: true
```

Conditional Statements

Conditional statements are used to make decisions in a program. They allow different actions to be executed depending on whether a condition is true or false. In Dart, conditional logic is implemented with if, else if and else. In this project the same pattern decides whether a farm needs water.

Code:

```
void main() {
  double moisture = 28;
  double minimum = 35;

  if (moisture < minimum - 10) {
    print('Water now');
  } else if (moisture < minimum) {
    print('Needs water');
  } else {
    print('Healthy');
  }
}
```

Output:

```
Needs water
```

Loops

Loops are used to execute a block of code repeatedly as long as a specified condition is true. They help automate repetitive tasks and reduce redundancy. Dart supports for, while and do-while loops.

- **for loop:** Repeats code a fixed number of times.
- **while loop:** Repeats while a condition is true.
- **do-while loop:** Executes the code once, then checks the condition.

Code:

```
void main() {
  List<String> crops = ['Rice', 'Tomato', 'Cotton'];

  for (int i = 0; i < crops.length; i++) {
    print('Crop ${i + 1}: ${crops[i]}');
  }
}
```

Output:

```
Crop 1: Rice
Crop 2: Tomato
Crop 3: Cotton
```

Lists

A List in Dart is an ordered collection of elements. Lists are similar to arrays in other programming languages. They can store multiple values of the same type and allow random access using an index. The dashboard uses lists to hold the farms, crops, alerts and soil readings that are loaded from the database.

Code:

```
void main() {
  List<String> farms = ['Green Farm', 'River Field', 'Hill Plot'];
  farms.add('Sunrise Farm');
  farms.remove('Hill Plot');
```

```
  print('Farms: $farms');
  print('Total farms: ${farms.length}');
}
```

Output:

```
Farms: [Green Farm, River Field, Sunrise Farm]
Total farms: 3
```

Maps

Maps in Dart store data as key-value pairs, where each key is unique. They are similar to dictionaries in Python or hash maps in Java and allow fast data retrieval using keys. SQLite rows are read as maps and are converted into model objects with a fromMap() constructor.

Code:

```
void main() {
  Map<String, double> moisture = {
    'Green Farm': 42,
    'River Field': 31,
    'Hill Plot': 26,
  };

  moisture.forEach((farm, value) {
    print('$farm: $value%');
  });
}
```

Output:

```
Green Farm: 42.0%
River Field: 31.0%
Hill Plot: 26.0%
```

Functions

Functions are reusable blocks of code designed to perform specific tasks. They make programs modular and easier to maintain. In Dart, functions can take positional and named parameters and can return values.

Code:

```
double waterNeed(double et0, double coefficient) {
  return et0 * coefficient;
}

void main() {
  double need = waterNeed(5.2, 1.15);
  print('Crop water need: ${need.toStringAsFixed(1)} mm');
}
```

Output:

```
Crop water need: 6.0 mm
```

Classes and Models

Dart is class-based, and every model in this project is a class. The following class stores one soil reading. It shows named constructor parameters, final fields and computed getters.

```
class SoilReading {
  const SoilReading({
    required this.moisturePercent,
    required this.minMoisturePercent,
  });

  final double moisturePercent;
  final double minMoisturePercent;

  bool get isMoistureLow => moisturePercent < minMoisturePercent;
}

void main() {
  const reading = SoilReading(
    moisturePercent: 28,
    minMoisturePercent: 35,
  );
  print('Low moisture: ${reading.isMoistureLow}');
}
```

Output:

```
Low moisture: true
```

Asynchronous Programming

Database queries, GPS readings and HTTP requests take time. Dart handles them with Future, async and await so that the user interface is not blocked while the work is in progress. Errors are caught with try, catch and finally.

```
Future<void> loadWeather() async {
  try {
    final location = await LocationService.getCurrentLocation();
    final weather = await WeatherService.fetchWeather(
    latitude: location.latitude,
      longitude: location.longitude,
    );
    print('Temperature: ${weather.temperature} C');
  } catch (e) {
    print('Weather error: $e');
  } finally {
    print('Weather request finished');
  }
}
```

9. Case Study on Smart Agriculture Monitoring App

This case study describes how the **Smart Agriculture Monitoring App** was designed and built for a small farmer who owns three plots of land and grows rice, tomato and cotton. The farmer already uses a smartphone, but all farm information is kept in a notebook, irrigation is decided by habit, and weather information comes from television or from neighbours. The aim of the case study is to show how a single Flutter application can replace the notebook and give the farmer clear, timely information about the farm.

Existing System

- Farm details such as plot size, soil type and main crop are written in a notebook that can be lost or damaged.
- The same crop may be sown on two plots on different dates, but there is no record of the planting date or of the growth stage.
- Irrigation is applied on a fixed weekly schedule, so water is wasted in the rainy season and crops are under-watered in hot weather.
- Soil moisture and water level are never measured, so the farmer cannot see a problem until the leaves start to wilt.
- Weather is known only from a general forecast that does not use the latitude and longitude of the farm.
- There is no reminder when moisture becomes low, and no record of crop diseases or of the advice that was followed.

Proposed System

The proposed system is a mobile application that stores every farm record in a local SQLite database and combines it with live weather data from the Open-Meteo API. Each time the farmer opens the dashboard, the application reads the farms, crops, alerts and the latest soil readings, fetches the current weather for the GPS position of the phone, estimates the daily water loss of the crop and raises an alert if the soil is too dry. The farmer can then record a new moisture reading, follow the irrigation advice given in litres, and read a short explanation of what the reading means.

Objectives of the Case Study

- To provide a secure login and registration facility so that the records of each farmer are kept separately.
- To allow a farmer to add, edit and delete farms and to see all the farms in one list.
- To keep crop details such as crop name, planting date, growth stage and status, and to show the age of each crop in days.
- To record soil moisture and water level readings and to keep the history of every reading for later comparison.
- To calculate the daily water requirement of a crop from the reference evapotranspiration, the crop coefficient and the area of the farm, and to express the result in litres.
- To show the current weather, the maximum and minimum temperature, the rain chance and a small forecast for the location of the farm.

- To generate smart alerts automatically when moisture is low, when the water level is low or when heavy rain is expected.
- To answer common farming questions offline and to store the history of the conversation.
- To scan a crop photograph, classify the visible condition and keep the scan in a history list.

Features of the Application

- **User accounts:** registration with name, email, phone and password, followed by login. The password is checked against the users table and the session is stored with shared_preferences.
- **Dashboard:** a greeting header, a live weather card, three counters (farms, crops and unread alerts), a soil and water summary with two gauges, quick action tiles, the latest alerts, the latest crop scans and a banner for the agricultural assistant.
- **Farm management:** a list of farms with size, soil type and main crop, a form to add or edit a farm, and a delete option with a confirmation dialog.
- **Crop monitoring:** crops are linked to a farm and store the planting date, growth stage and status. The screen groups the crops by farm and shows the number of days since planting.
- **Soil and water:** a reading form for moisture percentage, water level, water used and a note. The service computes the minimum moisture for the crop, the litres required and a status such as Healthy, Needs Water or Critical.
- **Weather:** current temperature, humidity, wind speed, rain chance and daily maximum and minimum values for the GPS position, with a refresh button and a fallback message when the network is unavailable.
- **Alerts:** alerts are generated by the application itself, stored in the alerts table, marked as read when opened and removed when the farmer clears them.
- **Assistant:** a chat screen that answers questions about irrigation, fertiliser, pests and crop selection, with voice input through speech_to_text and spoken answers through flutter_tts.
- **Crop scanner:** the camera or gallery is used to select a photograph, the condition is analysed and the result, the problem and the care advice are saved in the crop_scans table.

Technologies Used

Layer	Technology	Role in the application
User interface	Flutter 3.44, Material 3	All screens, widgets, navigation and animation
Language	Dart 3.12	Business logic, models and asynchronous code
Local database	SQLite through sqflite	Users, farms, crops, readings, alerts and scans
Desktop and web	sqflite_common_ffi, ffi_web	The same database API on Windows and in the browser
Session	shared_preferences	Keeps the id of the logged-in farmer
Weather	Open-Meteo REST API over http	Current weather and daily forecast

Layer	Technology	Role in the application
Location	geolocator, permission_handler	Latitude and longitude of the farm
Assistant	speech_to_text, flutter_tts	Voice input and spoken output
Images	image_picker	Camera and gallery access for crop scans
Testing	flutter_test, sqflite_common_ffi	Unit and widget tests on the desktop

Case Study Summary

The finished application turns the notebook of the farmer into a small database that is always available on the phone. Because the records are stored locally, the farm list, the crops and the readings can be opened without a network connection; only the weather card and the location need the internet. The dashboard shown below is the first screen after login and gives the farmer the complete state of the farm in one view.

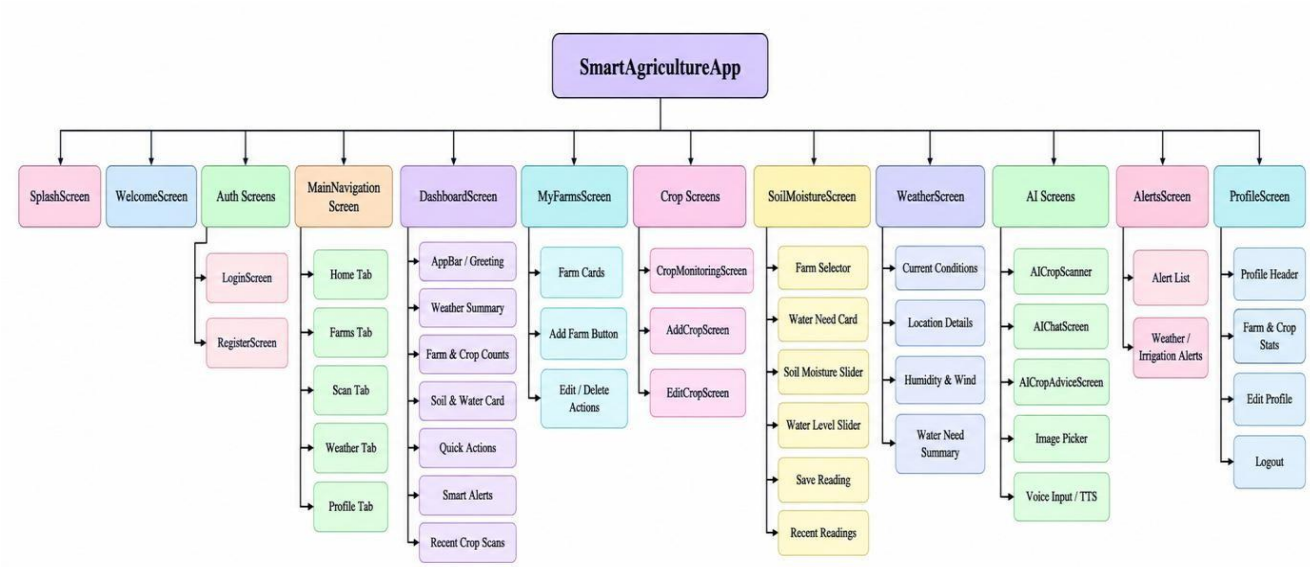


Main/Dashboard Screen

10. Exploring Flutter Widgets

In Flutter everything that appears on the screen is a widget. A widget is an immutable description of a part of the user interface; it does not draw anything itself, but it is inflated into an element and a render object that perform the layout and the painting. Widgets are combined into a tree, and a screen is simply the root of a small tree of widgets.

Widgets are divided into two broad groups. **Layout widgets** such as Row, Column, Stack, Container, Padding, Expanded and GridView decide where the children are placed and how much space they receive. **Presentation widgets** such as Text, Icon, Image, Card, ElevatedButton and TextField draw the visible content and react to the user.



Simplified Widget Tree of the Smart Agriculture Monitoring App

Building a Widget Tree

The following stateless widget shows how a Container, a Column and a Row are combined to build a small information card. The same structure is used by the farm cards and the crop cards of the application.

```
class InfoCard extends StatelessWidget {
  const InfoCard({super.key, required this.title, required this.value});

  final String title;
  final String value;

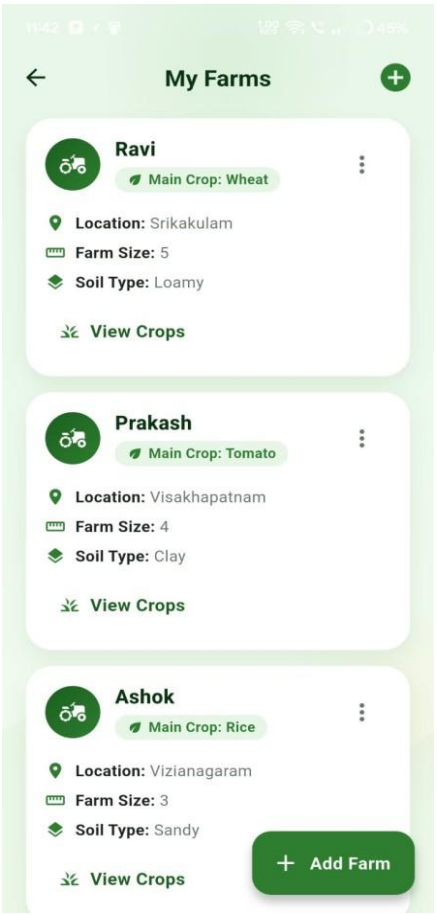
  @override
  Widget build(BuildContext context) {
    return Container(
      padding: const EdgeInsets.all(16),
      decoration: BoxDecoration(
      color: Colors.white,
        borderRadius: BorderRadius.circular(20),
      ),
```

```
      child: Row(
        children: [
          const Icon(Icons.agriculture, color: Color(0xFF2E7D32)),
          const SizedBox(width: 12),
          Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Text(title, style: const TextStyle(fontSize: 12)),
              Text(
                value,
                style: const TextStyle(
                  fontSize: 18,
                  fontWeight: FontWeight.w700,
                ),
              ),
            ],
          ),
        ],
      ),
    );
  }
}
```

The widget is then used like any built-in widget, and it can be placed anywhere in the tree:

```
InfoCard(title: 'Total Farms', value: '3')
```

Inside _statCard, a FittedBox with BoxFit.scaleDown is used around the number so that a large value still fits in the circle when the text scale of the phone is increased.



My Farms Screen

11. Responsive UI Design

A Flutter application can run on a phone with a 320 pixel wide screen, on a tablet in landscape orientation and on a Windows desktop window that can be resized freely. Responsive design means that the same widget tree adapts to the available space instead of assuming a fixed screen size. In this project the layout is adjusted with MediaQuery, LayoutBuilder, Expanded, FittedBox, ConstrainedBox and scrollable containers, and no screen uses hard-coded pixel positions.

LayoutBuilder

LayoutBuilder gives the constraints of the parent to its builder, so a widget can choose a different layout when more width is available. The quick action grid on the dashboard shows four tiles per row on a tablet or a desktop window and three tiles per row on a phone:

```
LayoutBuilder(
  builder: (context, constraints) {
    final columns = constraints.maxWidth >= 600 ? 4 : 3;
    return GridView.builder(
      shrinkWrap: true,
      physics: const NeverScrollableScrollPhysics(),
      itemCount: actions.length,
      gridDelegate: SliverGridDelegateWithFixedCrossAxisCount(
        crossAxisCount: columns,
        mainAxisSpacing: 10,
        crossAxisSpacing: 10,
        mainAxisExtent: 104,
      ),
      itemBuilder: (context, index) {
        final action = actions[index];
        return _actionTile(
          icon: action.$1,
          label: action.$2,
          gradient: action.$3,
          onTap: () => Navigator.of(context).push(
            MaterialPageRoute(builder: (_) => action.$4),
          ),
        );
      },
    );
  },
)
```

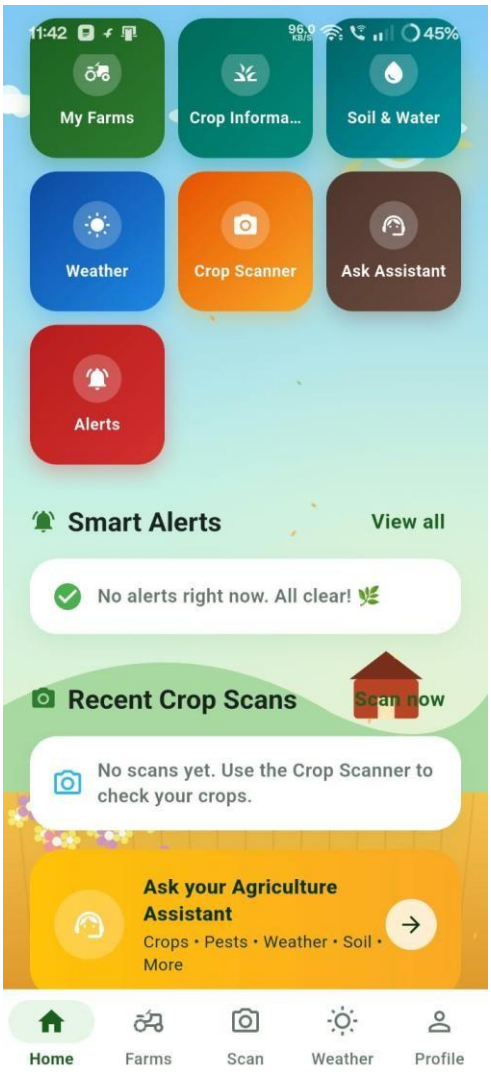
Flexible Layouts

Three more techniques are used throughout the screens:

1. **Expanded and Flexible:** divide the free space of a Row or Column. The three counters of the dashboard and the two gauges of the soil card share the width equally.
2. **FittedBox with BoxFit.scaleDown:** shrinks a large number or a long label instead of overflowing its container. It is used in the counter cards and in the soil gauges.
3. **ConstrainedBox:** limits the width of a form. The login, register and farm forms are placed inside a ConstrainedBox with maxWidth 460 so that the fields do not stretch across a desktop window.

Responsive Techniques Used

Technique	Effect	Where it is used
LayoutBuilder	Chooses 3 or 4 grid columns from the width	Dashboard quick actions
MediaQuery.sizeOf	Selects the compact splash layout	Splash screen
Expanded	Shares the row width between children	Counters, gauges, form buttons
FittedBox	Scales text down to avoid overflow	Counters, soil gauges
ConstrainedBox	Caps the width of a form at 460 px	Login, register, farm and crop forms
SingleChildScrollView	Allows the page to scroll	All forms
ListView / GridView	Scrolls long content	Dashboard, farms, crops, alerts



Quick action grid arranged by LayoutBuilder

12. Navigation Between Screens

Navigation is the movement of the user from one screen to another. Flutter manages screens as routes on a stack that is owned by the Navigator of the MaterialApp. A new screen is added to the top of the stack with Navigator.push, the top screen is removed with Navigator.pop, and the whole stack can be replaced when the user logs in or logs out.

Types of Navigation Used

- **Navigator.push:** opens a detail screen above the current screen and keeps the back button working. It is used for the farm form, the crop form, the soil reading form, the assistant and the alerts.
- **Navigator.pop:** closes the current screen and returns to the previous one, optionally with a result that tells the caller to reload its data.
- **Navigator.pushReplacement:** replaces the current route. The splash screen uses it so that the back button does not return to the splash screen.
- **Navigator.pushAndRemoveUntil:** clears the whole stack. Login and registration use it so that the dashboard becomes the only route and the back button cannot return to the login form.

Bottom Navigation

The five main sections of the application are kept in a MainNavigationScreen. An IndexedStack holds all five screens at the same time and shows only the selected one, so the scroll position and the loaded data of each tab are preserved when the farmer switches between them.

```
class _MainNavigationScreenState extends State<MainNavigationScreen> {
 int _index = 0;


 @override
 Widget build(BuildContext context) {
  return Scaffold(
   backgroundColor: AppColors.background,
   body: IndexedStack(
    index: _index,
    children: const [
    DashboardScreen(),
    MyFarmsScreen(),
     AICropScannerScreen(),
     WeatherScreen(),
     ProfileScreen(),
    ],
   ),
   bottomNavigationBar: NavigationBar(
    selectedIndex: _index,
    onDestinationSelected: (index) =>
       setState(() => _index = index),
    destinations: const [
    NavigationDestination(
       icon: Icon(Icons.home_outlined),
       selectedIcon: Icon(Icons.home),
       label: 'Home',
     ),
```

```
        NavigationDestination(
          icon: Icon(Icons.agriculture_outlined),
          selectedIcon: Icon(Icons.agriculture),
          label: 'Farms',
        ),
        NavigationDestination(
          icon: Icon(Icons.photo_camera_outlined),
          selectedIcon: Icon(Icons.photo_camera),
          label: 'Scan',
        ),
        NavigationDestination(
          icon: Icon(Icons.wb_sunny_outlined),
          selectedIcon: Icon(Icons.wb_sunny),
          label: 'Weather',
        ),
        NavigationDestination(
          icon: Icon(Icons.person_outline),
          selectedIcon: Icon(Icons.person),
          label: 'Profile',
        ),
      ],
    ),
  );
 }
}
```

Passing a Route

The dashboard opens the farm form with a MaterialPageRoute. The result of the route is awaited, and when the form returns true the farm list and the counters are refreshed:

```
onTap: () async {
 final saved = await Navigator.of(context).push<bool>(
  MaterialPageRoute(builder: (_) => const AddEditFarmScreen()),
 );
 if (saved == true) {
  await _load();
 }
}
```

After a successful login the stack is cleared so that the dashboard cannot be left with the back button:

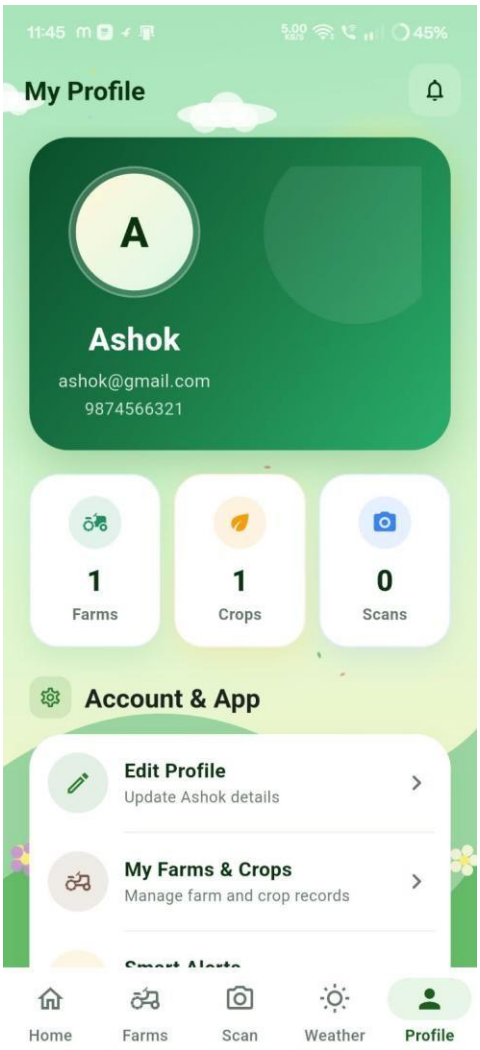
```
Navigator.of(context).pushAndRemoveUntil(
 MaterialPageRoute(builder: ( ) => const MainNavigationScreen()),
 (route) => false,
);
```

Navigation Map

The complete flow of the application is shown below. Each arrow represents a route pushed by the application.

```
SplashScreen
  -> WelcomeScreen              (pushReplacement, after 4 seconds)
      -> LoginScreen            (push)
          -> RegisterScreen     (push)
          -> MainNavigationScreen (pushAndRemoveUntil)
      -> RegisterScreen
          -> MainNavigationScreen (pushAndRemoveUntil)

MainNavigationScreen  (IndexedStack + NavigationBar)
  |-- DashboardScreen
  |    -> MyFarmsScreen -> AddEditFarmScreen
  |    -> CropMonitoringScreen -> AddEditCropScreen
  |    -> SoilMoistureScreen
  |    -> WeatherScreen
  |    -> AICropScannerScreen
  |    -> AIChatScreen
  |    -> AlertsScreen
  |-- MyFarmsScreen
  |-- AICropScannerScreen
  |-- WeatherScreen
  |-- ProfileScreen -> DatabaseViewerScreen (debug)
```



Profile Screen

13. Stateful and Stateless Widgets

A widget is either stateless or stateful. The difference is whether the widget owns data that can change while the application is running. Choosing the correct type keeps the widget tree small and prevents unnecessary rebuilds.

Stateless Widget

A StatelessWidget describes a part of the interface that depends only on its constructor arguments. It has a build method and no mutable fields, so it is rebuilt only when its parent rebuilds it with new values. Titles, icons, badges, cards without interaction and empty states are stateless in this project.

empty_state.dart

```
class EmptyState extends StatelessWidget {
  const EmptyState({
    super.key,
    required this.icon,
    required this.title,
    required this.message,
  });

  final IconData icon;
  final String title;
  final String message;

  @override
  Widget build(BuildContext context) {
    return Center(
      child: Padding(
        padding: const EdgeInsets.all(32),
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            Container(
              width: 92,
              height: 92,
              decoration: const BoxDecoration(
                shape: BoxShape.circle,
                color: AppColors.paleGreen,
              ),
              child: Icon(icon, size: 46, color: AppColors.green),
            ),
            const SizedBox(height: 18),
            Text(
              title,
              textAlign: TextAlign.center,
              style: Theme.of(context).textTheme.titleLarge,
            ),
            const SizedBox(height: 8),
            Text(
              message,
              textAlign: TextAlign.center,
              style: Theme.of(context).textTheme.bodyMedium,
            ),
          ],
```

```
      ),
    ),
  );
 }
}
```

Stateful Widget

A StatefulWidget delegates its interface to a State object that survives between rebuilds. The State holds the fields that change, such as the loaded lists and the loading flags, and calls setState when a value changes so that Flutter schedules a rebuild of that subtree. The dashboard is stateful because it loads data from the database and from the weather API.

dashboard_screen.dart

```
class DashboardScreen extends StatefulWidget {
  const DashboardScreen({super.key});

  @override
  State<DashboardScreen> createState() => _DashboardScreenState();
}

class _DashboardScreenState extends State<DashboardScreen> {
  bool _loading = true;
  int _farmCount = 0;
  int _cropCount = 0;
  int _unreadAlerts = 0;
  List<Alert> _alerts = [];
  List<CropScan> _scans = [];
  List<SoilReading> _soilReadings = [];
  WeatherData? _weather;

  @override
  void initState() {
    super.initState();
    _load();
  }

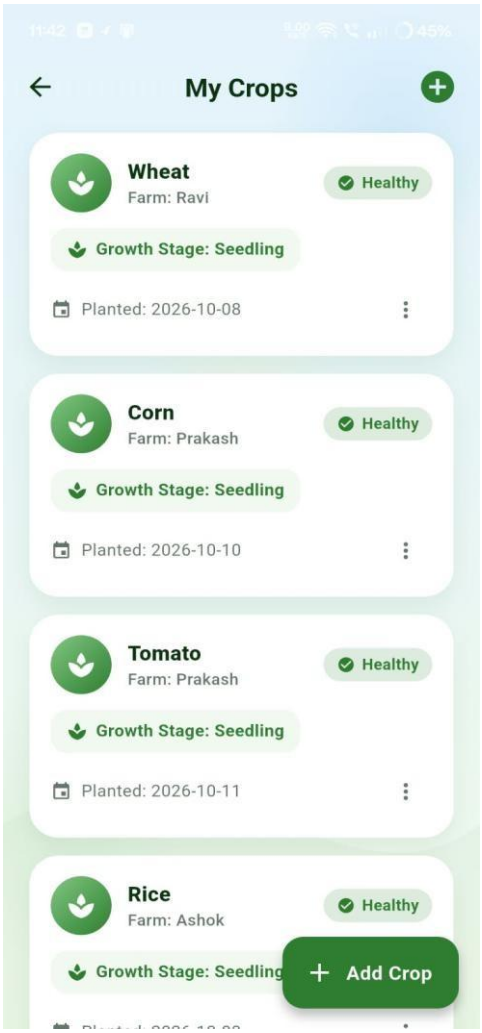
  Future<void> _load() async {
    final user = SessionService.instance.currentUser;
    if (user == null) {
      setState(() => _loading = false);
      return;
    }
    final farms = await DatabaseHelper.instance.getAllFarms(user.id!);
    final crops = await DatabaseHelper.instance.getCropsForUser(user.id!);
    final alerts = await DatabaseHelper.instance.getAlerts(user.id!);
    if (!mounted) return;
    setState(() {
      _farmCount = farms.length;
      _cropCount = crops.length;
      _alerts = alerts;
      _unreadAlerts = alerts.where((a) => !a.isRead).length;
      _loading = false;
    });
  }
```

Lifecycle of a StatefulWidget

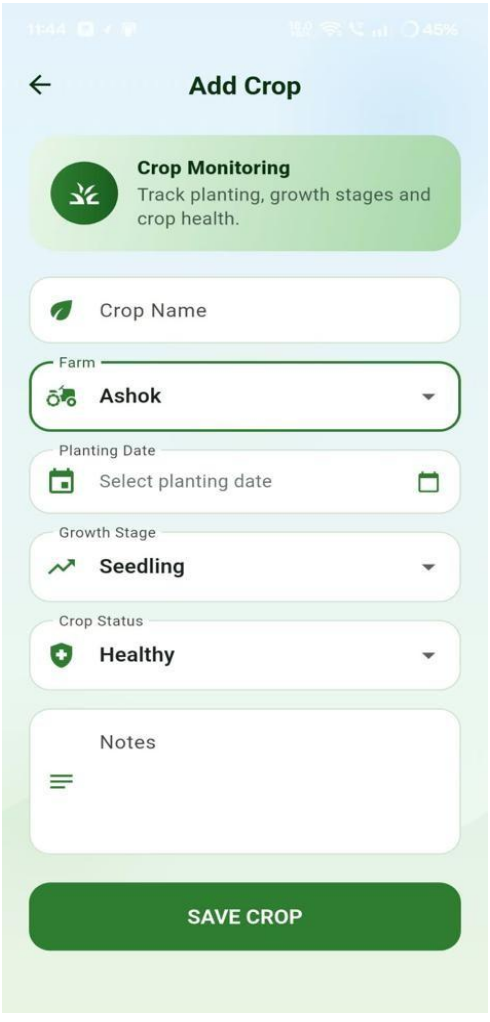
1. **createState()** creates the State object for the widget.
2. **initState()** runs once; the dashboard starts its database load here.
3. **didChangeDependencies()** runs when an inherited widget changes.
4. **build()** describes the interface and runs after every setState.
5. **didUpdateWidget()** runs when the parent passes a new configuration.

Comparison

Point	StatelessWidget	StatefulWidget
Mutable data	None	Held in the State object
Rebuild	When the parent rebuilds	When setState is called
Lifecycle	Only build()	initState, build, dispose and others
Typical use	Labels, cards, empty states	Forms, lists loaded from a database
Example	EmptyState, InfoCard	DashboardScreen, LoginScreen



Crop Monitoring Screen



Add / Edit Crop Screen

14. Custom Widgets and Styling

A custom widget is a class that groups several built-in widgets into one reusable component. Instead of repeating the same Container, padding, radius and shadow in every screen, the project defines the widget once and configures it with constructor parameters. This reduces duplication, keeps the screens short and makes the appearance of the application consistent.

Custom Widgets of the Project

Widget	Type	Responsibility
EmptyState	StatelessWidget	Icon, title and message shown when a list is empty
FarmBackground	StatelessWidget	Paints the sky and hill landscape behind the farm screens
AnimatedBackground	StatefulWidget + CustomPainter	Animated sky, clouds and particles for the welcome screens
DashboardBackground	StatefulWidget + CustomPainter	Slowly moving farm scene behind the dashboard
AuthBackground	StatelessWidget	Background shared by the login and register screens
AlertStyle	Utility class	Icon, colour and label for each alert type
DateFormats	Utility class	Formats dates as "Monday, 12 October 2026" and as "2h ago"

Colour Palette

All colours are declared once in the AppColors class. The palette is based on agricultural green, sky blue, earth brown and amber, and every screen refers to these constants instead of writing a colour literal.

```
class AppColors {
  AppColors._();

  static const Color primary = Color(0xFF1B5E20);
  static const Color primaryDark = Color(0xFF0C3311);
  static const Color green = Color(0xFF2E7D32);
  static const Color lightGreen = Color(0xFF81C784);
  static const Color paleGreen = Color(0xFFE8F5E9);
  static const Color skyBlue = Color(0xFF29B6F6);
  static const Color earth = Color(0xFF795548);
  static const Color amber = Color(0xFFFFC107);
  static const Color danger = Color(0xFFD32F2F);
  static const Color background = Color(0xFFF4FAF4);
  static const Color textDark = Color(0xFF1B231E);
  static const Color textGrey = Color(0xFF64756B);
}
```

Theming

The theme is built once in buildAppTheme() and applied to the MaterialApp. It sets the Material 3 colour scheme, the text styles, the transparent app bar, the 20 pixel card radius, the rounded input fields, the 54 pixel high buttons, the navigation bar and the floating snack bar. A screen therefore only writes semantic colours such as AppColors.green, and a change to the theme updates the whole application.

```
ThemeData buildAppTheme() {
  final base = ThemeData(
  useMaterial3: true,
    colorScheme: ColorScheme.fromSeed(
      seedColor: AppColors.primary,
      primary: AppColors.primary,
      secondary: AppColors.skyBlue,
      surface: AppColors.white,
      error: AppColors.danger,
    ),
    scaffoldBackgroundColor: AppColors.background,
  );
  return base.copyWith(
    cardTheme: const CardThemeData(
      elevation: 0,
      color: AppColors.white,
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.all(Radius.circular(20)),
      ),
    ),
    elevatedButtonTheme: ElevatedButtonThemeData(
      style: ElevatedButton.styleFrom(
      backgroundColor: AppColors.green,
      minimumSize: const Size.fromHeight(54),
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(16),
        ),
      ),
    ),
  );
}
```

Custom Painting

The landscapes behind the screens are not images: they are drawn at runtime by a CustomPainter. FarmLandscapePainter fills a gradient sky, adds soft radial blobs for the sun and the clouds and then draws two hills with a quadratic path. Because the drawing uses the size of the canvas, it fits every screen without a separate asset for each resolution.

```
class FarmLandscapePainter extends CustomPainter {
  const FarmLandscapePainter({this.scene = FarmScene.sunny});


  final FarmScene scene;


  @override
  void paint(Canvas canvas, Size size) {
    final w = size.width;
    final h = size.height;
```

```
    if (w <= 0 || h <= 0) return;
    final s = _specs[scene]!;

    _paintGradient(canvas, w, h, s);
    _paintBlobs(canvas, w, h, s);
    _paintHill(canvas, w, h, s);
  }

  @override
  bool shouldRepaint(FarmLandscapePainter oldDelegate) =>
      oldDelegate.scene != scene;
}
```

Formatting Helper

DateFormats is a small utility class with static methods. It removes the need to import the intl package and gives the same date format to the dashboard, the alerts and the scan history.

```
static String pretty(DateTime dt) {
  return '${_days[dt.weekday - 1]}, '
      '${dt.day} ${_months[dt.month - 1]} ${dt.year}';
}

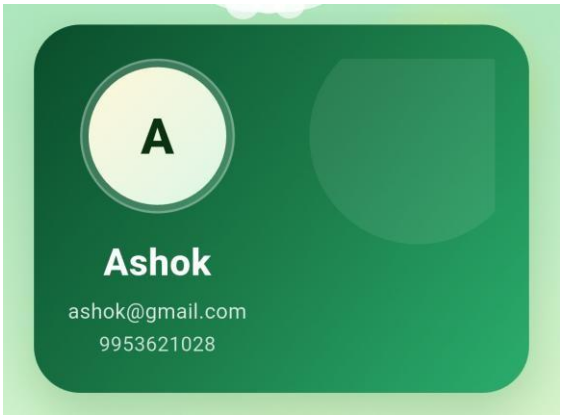
static String timeAgo(String iso) {
  final dt = DateTime.parse(iso);
  final diff = DateTime.now().difference(dt);
  if (diff.inMinutes < 1) return 'Just now';
  if (diff.inMinutes < 60) return '${diff.inMinutes}m ago';
  if (diff.inHours < 24) return '${diff.inHours}h ago';
  if (diff.inDays < 7) return '${diff.inDays}d ago';
  return short(dt);
}
```

Application Icon

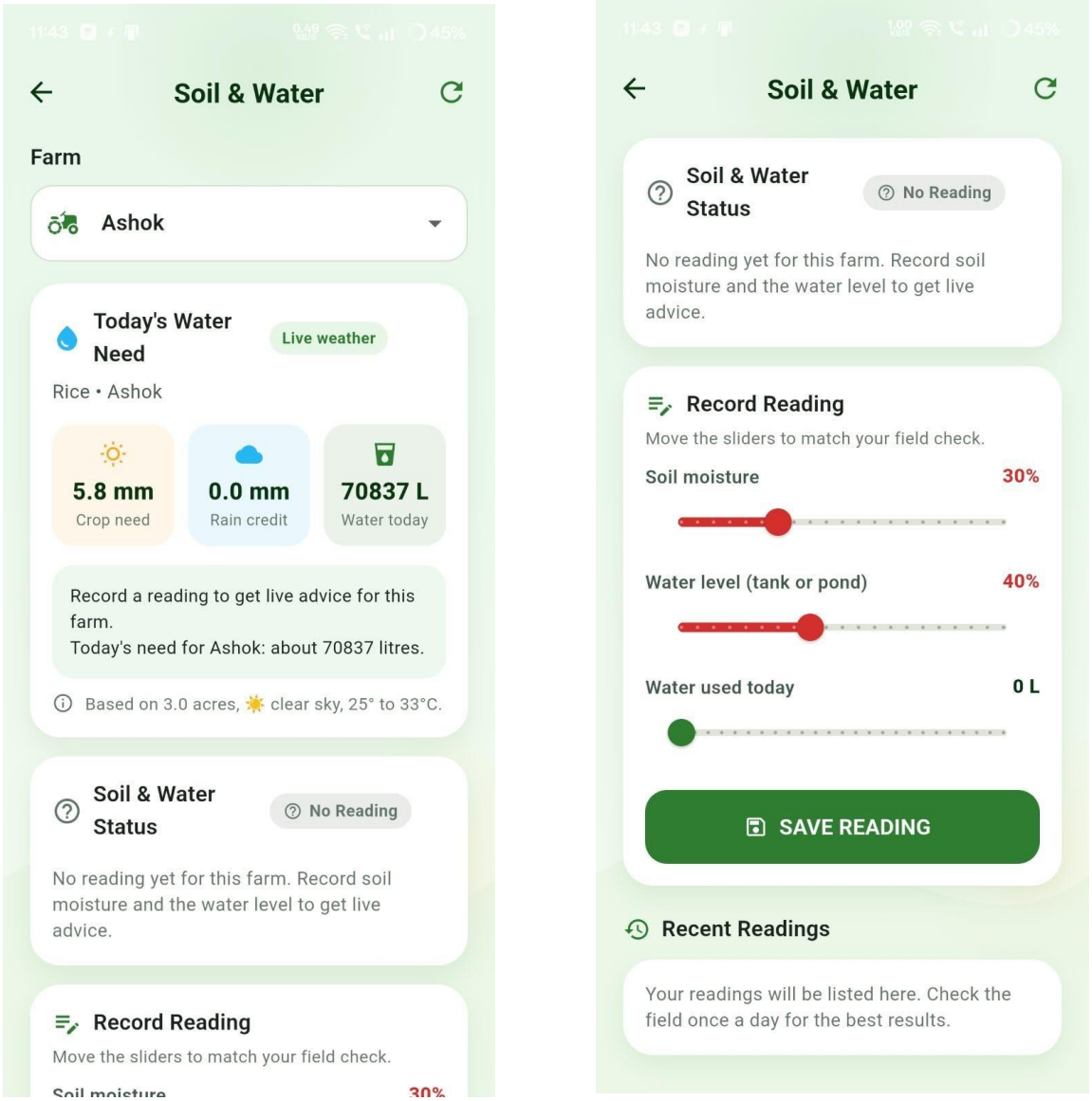
The launcher icon is a single 1254 x 1254 PNG file. The flutter_launcher_icons package generates all the Android and iOS icon sizes from this file, so the same artwork is used on the splash screen, in the app drawer and on the home screen of the phone.



App Icon Widget



Profile Widget



Soil Moisture and Water Management Screen

15. Forms and Validation

A form is the main way in which a user enters data. Flutter provides the Form widget, which groups several FormField widgets and validates them together, and TextFormField, which is a text field with an optional validator. In this application every piece of data that is written to the database passes through a form: the login form, the registration form, the farm form, the crop form, the soil reading form and the profile form.

Form Widgets Used

- **Form:** owns a GlobalKey<FormState> and validates all of its children when validate() is called.
- **TextFormField:** combines a TextField with a FormField so that a validator function can check the value.
- **InputDecoration:** adds a floating label, a hint, a prefix icon and the eye icon that shows or hides a password.
- **validator:** returns an error message, which is displayed under the field, or null when the value is correct.
- **autovalidateMode:** set to AutovalidateMode.onUserInteraction so that a field is checked again as soon as the user edits it.
- **keyboardType and textInputAction:** select the correct keyboard and the Next or Done key on the soft keyboard.

Validation Rules

Field	Rule	Error message
Full Name	Must not be empty	Please enter your full name
Email	Must match ^[\w\.\-+]+@[\w\-]+(\.[\w\-]+)+$	Please enter a valid email
Phone	Must match ^[+]?[\d\s\-]{10,15}$	Enter a valid phone number
Password	At least 6 characters	Password must be at least 6 characters
Confirm Password	Must equal the password field	Passwords do not match
Farm Name	Must not be empty	Please enter a farm name
Moisture / Water level	Required, between 0 and 100	Enter a value between 0 and 100

Login Form

The email field uses a regular expression that accepts the usual local part, the at sign and a domain with at least one dot. The password field only checks that a value was entered, because the correct password is verified against the database.

```
final _formKey = GlobalKey<FormState>();
final _emailController = TextEditingController();
final _passwordController = TextEditingController();
```

```
static final RegExp _emailRegExp = RegExp(
  r'^[\w\.\-+]+@[\w\-]+(\.[\w\-]+)+$',
);

TextFormField(
  controller: _emailController,
  keyboardType: TextInputType.emailAddress,
  textInputAction: TextInputAction.next,
  autovalidateMode: AutovalidateMode.onUserInteraction,
  decoration: const InputDecoration(
    labelText: 'Email',
    prefixIcon: Icon(Icons.mail_outline),
  ),
  validator: (value) {
    final v = value?.trim() ?? '';
    if (v.isEmpty) return 'Please enter your email';
    if (!_emailRegExp.hasMatch(v)) {
      return 'Please enter a valid email';
    }
    return null;
  },
),
```

Registration Form

The registration form adds a name field, a phone field and a confirmation field. The confirmation validator compares the value with the password controller, so the two passwords must be exactly the same before the account is created.

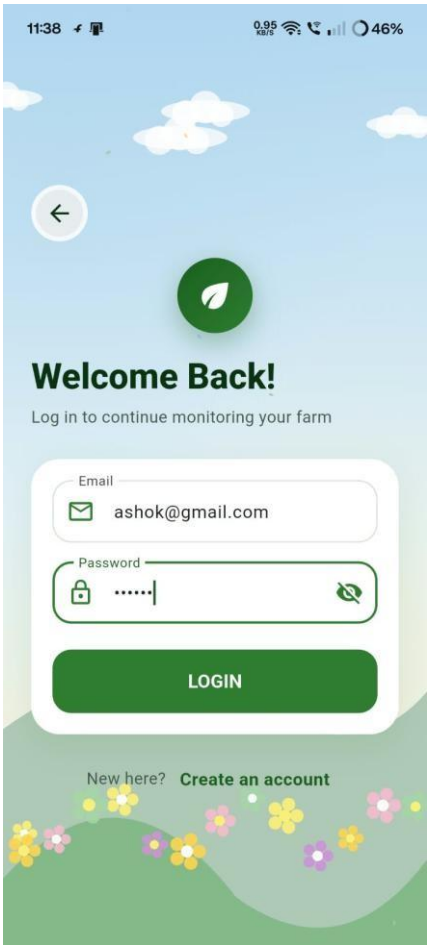
```
validator: (value) {
  if ((value ?? '').isEmpty) {
    return 'Please enter a password';
  }
  if ((value ?? '').length < 6) {
    return 'Password must be at least 6 characters';
  }
  return null;
},

// Confirm password field
validator: (value) {
  if ((value ?? '').isEmpty) {
    return 'Please confirm your password';
  }
  if (value != _passwordController.text) {
    return 'Passwords do not match';
  }
  return null;
},
```

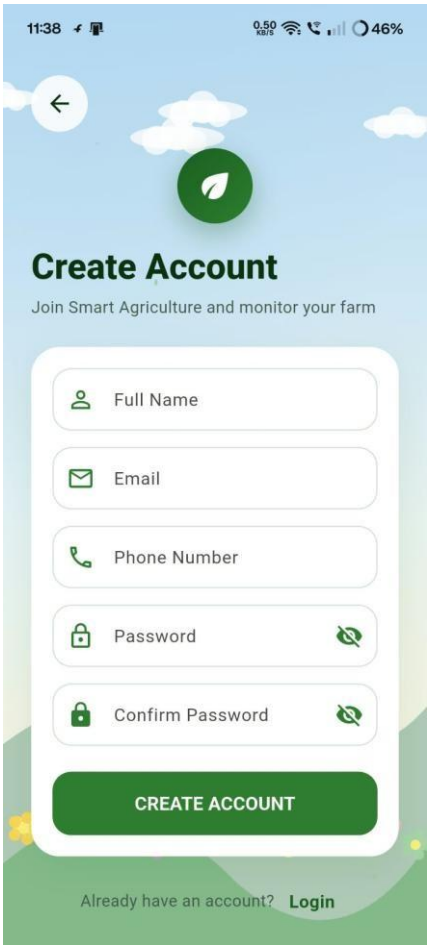
Submitting the Form

The submit method first validates the form. If any validator returns a message, the method returns immediately and the errors are shown under the fields. Otherwise the loading flag is set, the user is checked in the database, the session is saved and the navigation stack is replaced by the dashboard. A try, catch and finally block keeps the button disabled during the request and restores it afterwards, even when an error occurs.

```
Future<void> _submit() async {
  if (!_formKey.currentState!.validate()) return;
  setState(() => _loading = true);
  try {
    final user = await DatabaseHelper.instance.loginUser(
      email: _emailController.text.trim(),
      password: _passwordController.text,
    );
    await SessionService.instance.setUser(user);
    if (!mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Welcome back, farmer!')),
    );
    Navigator.of(context).pushAndRemoveUntil(
      MaterialPageRoute(builder: (_) => const MainNavigationScreen()),
      (route) => false,
    );
  } catch (e) {
    if (!mounted) return;
    final message = e.toString().replaceFirst('Exception: ', '');
    ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(content: Text(message)),
    );
  } finally {
    if (mounted) setState(() => _loading = false);
  }
}
```



Login Screen



Registration Screen

16. Animations

Animations make a mobile interface feel responsive and help the user understand what changed on the screen. Flutter runs animations on the same rendering pipeline as the rest of the interface, so a well-written animation stays smooth at 60 frames per second. The application uses animations in three places: the splash screen, the welcome pages and the painted backgrounds of the main screens.

Types of Animation

Type	How it works	Used in this project
Implicit	The widget animates to a new value automatically	Page indicator spacing on the welcome screen
Explicit	An AnimationController is driven by vsync and a Ticker	Splash screen, painted backgrounds
Tween	Maps the controller value from a begin value to an end value	Scale, fade and slide of the splash content
CurvedAnimation	Shapes the speed of the animation	easeIn, elasticOut, easeOutCubic
CustomPainter	Paints a frame from the current controller value	Sky, clouds, hills and particles

Splash Screen Animation

The splash screen is a StatefulWidget with a SingleTickerProviderStateMixin. A single controller of 1500 milliseconds drives three animations: a fade with Curves.easeIn, a scale from 0.65 to 1.0 with Curves.elasticOut and a slide from Offset(0, 0.25) to zero with Curves.easeOutCubic. The scale, fade and slide transitions are then applied to the application icon and to the title, and a timer of four seconds opens the welcome screen.

```
class _SplashScreenState extends State<SplashScreen>
    with SingleTickerProviderStateMixin {
  late final AnimationController _controller;
  late final Animation<double> _fadeAnimation;
  late final Animation<double> _scaleAnimation;
  late final Animation<Offset> _slideAnimation;

  @override
  void initState() {
    super.initState();

    _controller = AnimationController(
      vsync: this,
      duration: const Duration(milliseconds: 1500),
    );

    _fadeAnimation = CurvedAnimation(
      parent: _controller,
      curve: Curves.easeIn,
    );
```

```
  _scaleAnimation = Tween<double>(begin: 0.65, end: 1.0).animate(
    CurvedAnimation(parent: _controller, curve: Curves.elasticOut),
  );

  _slideAnimation = Tween<Offset>(
    begin: const Offset(0, 0.25),
    end: Offset.zero,
  ).animate(
    CurvedAnimation(parent: _controller, curve: Curves.easeOutCubic),
  );

  _controller.forward();

  Timer(const Duration(seconds: 4), () {
    if (!mounted) return;
    Navigator.of(context).pushReplacement(
      MaterialPageRoute(builder: (_) => const WelcomeScreen()),
    );
  });
 }
```

The animations are applied with the transition widgets, which rebuild only the animated part of the tree:

```
ScaleTransition(
  scale: _scaleAnimation,
  child: FadeTransition(
  opacity: _fadeAnimation,
    child: Image.asset('assets/images/app_icon.png'),
  ),
),
SlideTransition(
  position: _slideAnimation,
  child: FadeTransition(
  opacity: _fadeAnimation,
    child: const Text('SMART AGRICULTURE'),
  ),
),
```

Page Transition on the Welcome Screen

The welcome screen uses a PageController to move between three onboarding pages. The buttons call nextPage(), previousPage() or animateToPage() with a duration of 500 to 600 milliseconds and the Curves.easeInOutCubic curve, and onPageChanged updates the page indicator below the illustration.

```
final PageController _pageController = PageController();

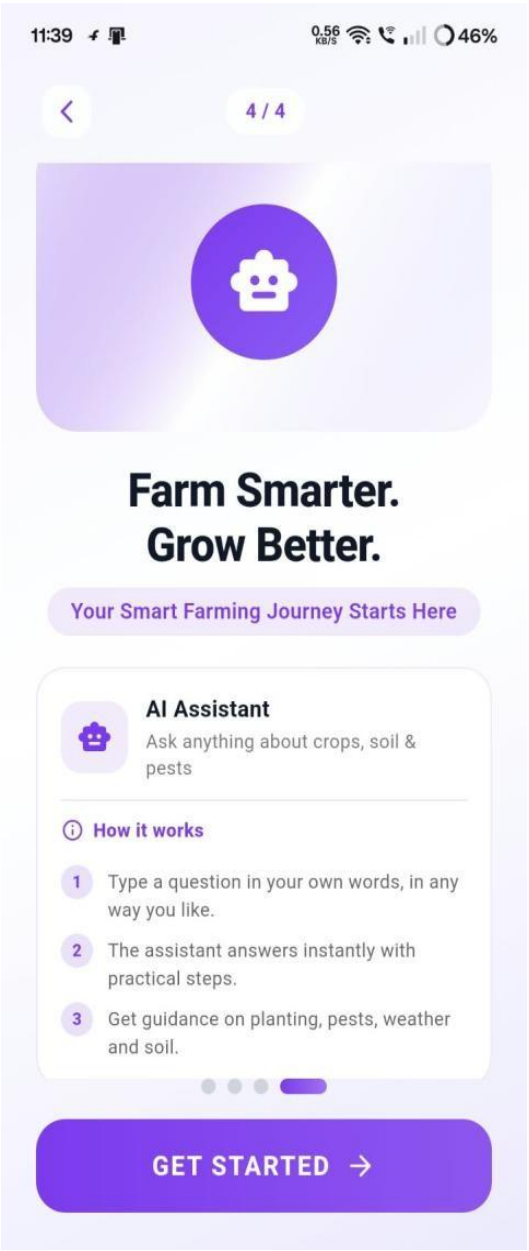
Expanded(
  child: PageView.builder(
    controller: _pageController,
    itemCount: _pages.length,
    onPageChanged: (index) {
    setState(() {
      _currentPage = index;
```

```
      });
    },
    itemBuilder: (context, index) {
      return _buildFullPage(_pages[index], size);
    },
  ),
),
```

Every controller is disposed in the dispose method, which releases the ticker and prevents a memory leak when the screen is removed.



Splash Screen



Onboarding Screen

17. REST API Integration

A REST API is a web service that exchanges data over HTTP in a stateless way. The client sends a request to an endpoint with a method such as GET, POST, PUT or DELETE, and the server answers with a status code and a body that is usually JSON. The Smart Agriculture Monitoring App uses the free **Open-Meteo** weather API, which needs no API key and returns the current weather and a daily forecast for a latitude and longitude.

Endpoint Used

Item	Value
Base URL	https://api.open-meteo.com/v1/forecast
Method	GET
Authentication	None (open access API)
Current fields	temperature_2m, relative_humidity_2m, weather_code, wind_speed_10m
Daily fields	temperature_2m_max, temperature_2m_min, precipitation_sum, precipitation_probability_max
Units	Celsius, km/h, millimetres
Timeout	20 seconds

The request is built with Uri.parse and replace, so the query parameters are encoded correctly and the latitude and longitude are rounded to four decimal places.

Calling the API

The http package sends the GET request asynchronously. A non-200 status code is converted into an exception with a message that can be shown to the farmer, and the JSON body is decoded into a Map<String, dynamic>. Only the fields that the application needs are read; every value is cast with a fallback so that a missing field does not crash the screen.

```
class WeatherService {
  WeatherService._();

  static const String _baseUrl =
      'https://api.open-meteo.com/v1/forecast';

  static Future<WeatherData> fetchWeather({
    required double latitude,
    required double longitude,
  }) async {
    final uri = Uri.parse(_baseUrl).replace(queryParameters: {...});

    final response = await http.get(uri).timeout(
      const Duration(seconds: 20),
    );
    if (response.statusCode != 200) {
      throw Exception(
        'Could not fetch weather. Please check your connection.',
      );
```

```
    }

    final json = jsonDecode(response.body) as Map<String, dynamic>;
    final current = json['current'] as Map<String, dynamic>?;
    if (current == null) {
      throw Exception('Weather data is not available right now.');
    }

    final daily = json['daily'] as Map<String, dynamic>?;

    return WeatherData(
      temperature:
          (current['temperature_2m'] as num?)?.toDouble() ?? 0,
      humidity:
          (current['relative_humidity_2m'] as num?)?.toDouble() ?? 0,
      windSpeed:
          (current['wind_speed_10m'] as num?)?.toDouble() ?? 0,
      weatherCode: (current['weather_code'] as num?)?.toInt() ?? 0,
      time: (current['time'] as String?) ?? '',
      latitude: latitude,
      longitude: longitude,
      dailyMaxTemp: _first(daily, 'temperature_2m_max'),
      dailyMinTemp: _first(daily, 'temperature_2m_min'),
      precipitationMm: _first(daily, 'precipitation_sum'),
      rainChancePercent:
          _first(daily, 'precipitation_probability_max'),
    );
  }
}
```

Sample Response

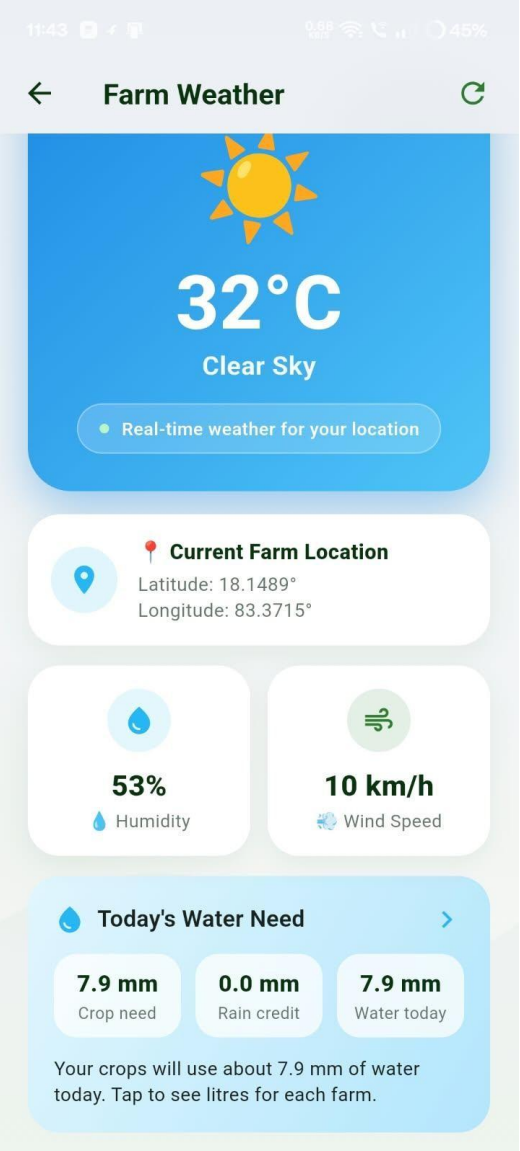
The server returns a JSON object with a current block and a daily block whose values are lists, because the same request can ask for several days. The application asks for one day and reads the first element of each list with the _first helper.


```
{
  "latitude": 17.4,
  "longitude": 78.5,
  "timezone": "Asia/Kolkata",
  "current": {
    "time": "2026-10-30T10:00",
    "temperature_2m": 29.4,
    "relative_humidity_2m": 58,
    "weather_code": 1,
    "wind_speed_10m": 9.7
  },
  "daily": {
    "time": ["2026-10-30"],
    "temperature_2m_max": [31.8],
    "temperature_2m_min": [21.2],
    "precipitation_sum": [0.0],
    "precipitation_probability_max":  [10]
  }
}
```

Displaying the Data

The WeatherData model converts the numeric weather code into a readable condition and an emoji, so the interface can show "Mainly Clear" with a sun and cloud symbol instead of the number 1. The weather screen also shows the humidity, the wind speed, the rain chance and the daily maximum and minimum, and a pull to refresh action repeats the request.

Weather code	Condition	Symbol
0	Clear Sky	Sun
1 / 2 / 3	Mainly Clear / Partly Cloudy / Overcast	Sun and cloud
45 / 48	Foggy	Fog
51 to 57	Drizzle / Freezing Drizzle	Drizzle
61 to 67	Rainy / Freezing Rain	Rain
80 to 82	Rain Showers	Shower
95 to 99	Thunderstorm	Storm



Live weather Screen

18. Testing and Debugging

Testing checks that the application behaves as expected, and debugging finds the reason when it does not. Flutter provides the flutter_test package for automated tests, the Flutter DevTools for inspecting the widget tree and the performance, and the debug console for log messages and exceptions.

Types of Tests

- **Unit tests** check a single function or class without a screen. In this project they test the farm area parser, the water balance, the thresholds and the dashboard helpers of SoilWaterService.
- **Widget tests** build a widget in a simulated environment and check what the user would see. They are used to render the dashboard and the soil screen at four screen widths.
- **Integration tests** run the whole application on a device; they are not part of this project because the database and the weather API are already covered by the two other levels.

Test Files

File	Cases	What it verifies
test/soil_water_service_test.dart	15	Area parsing, ET0, rain credit, thresholds, litres, plan
test/layout_test.dart	8	Dashboard and soil screen at 320, 360, 414 and 768 px
test/widget_test.dart	1	The splash screen at start-up

The calculator tests build small Farm, WeatherData and SoilReading objects and compare the result with the expected value. The following test checks that a hot dry day needs more water than a mild day, using the daily balance of the service.

```
test('a hot dry day needs more water than a mild day', () {
  final hot = SoilWaterService.dailyBalance(
    farm: _farm(),
    crop: null,
    weather: _weather(max: 40, min: 28, rain: 0),
  );
  final mild = SoilWaterService.dailyBalance(
    farm: _farm(),
    crop: null,
    weather: _weather(max: 28, min: 20, rain: 0),
  );

  expect(hot.netNeedMm, greaterThan(mild.netNeedMm));
});
```

The layout tests set the physical size of the test window, pump the screen and then check that no overflow exception was reported. The same test runs for every size in the list, so one declaration produces four test cases.

```
const List<Size> _screenSizes = [
  Size(320, 2400),
  Size(360, 2600),
  Size(414, 2800),
  Size(768, 3200),
];

testWidgets(
  'dashboard renders without overflow at ${size.width.toInt()}px',
  (tester) async {
  tester.platformDispatcher.textScaleFactorTestValue = 1.3;
  addTearDown(
      tester.platformDispatcher.clearTextScaleFactorTestValue,
    );

    await _pumpAt(tester, const DashboardScreen(), size);

    expect(find.text('Soil & Water'), findsWidgets);
    expect(tester.takeException(), isNull);
  },
);
```

Running the Tests

The tests are executed from the project folder with the following commands. The first command checks the source code for errors and style problems, and the second runs every test file.

```
flutter analyze
flutter test
```

The analyzer reports that the project has no issues. The test run executes 24 cases: the 15 calculator tests and the 8 layout tests pass, while the start-up widget test fails.

```
Analyzing smart_agriculture_app...
No issues found! (ran in 9.1s)


00:09 +23 -1: Some tests failed.
Failing tests:
  test/widget_test.dart: App shows the splash screen while it starts up
```

Debugging a Failing Test

The widget test expects the text "Smart Agriculture", but the splash screen displays the title in capitals as "SMART AGRICULTURE". The failure message is therefore clear: the finder found zero widgets with the expected text. The test is corrected by matching the text that the screen actually shows, or by making the expectation case-insensitive.

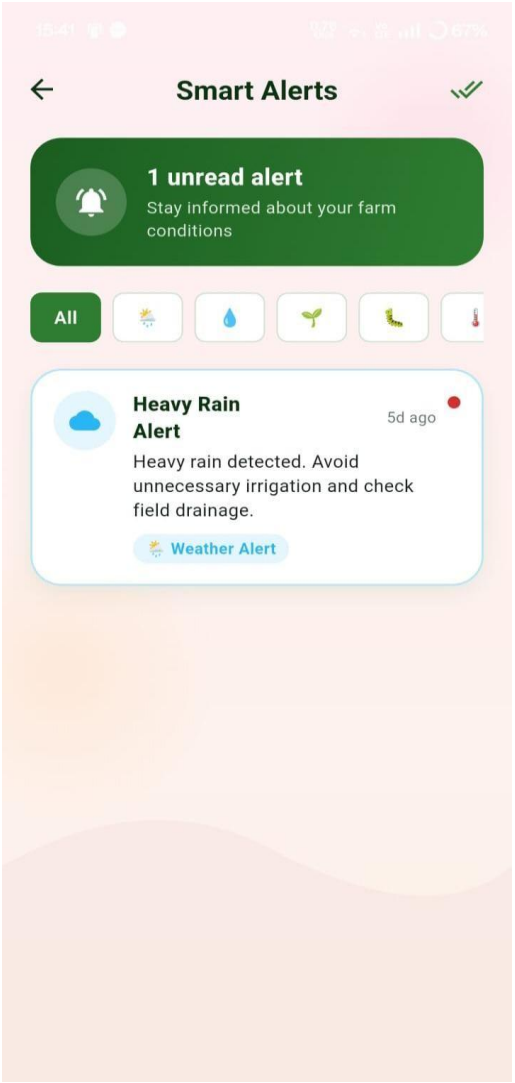
```
Expected: exactly one matching candidate
  Actual: _TextWidgetFinder:<Found 0 widgets with text
          "Smart Agriculture": []>
The test description was:
  App shows the splash screen while it starts up
```

A widget test reports the line of the failing expectation, which makes this kind of error quick to locate. Other common problems are found in the same way: an overflow reports the widget that

could not fit, a null check error reports the field that was not initialised, and a database error reports the SQL statement that failed.

Debugging Techniques Used

- **print statements:** main() prints "DIAG main start", "DIAG factory ok" or "DIAG factory FAILED" so that the database initialisation can be checked in the console of any platform.
- **Breakpoints:** a breakpoint in VS Code pauses the application at a line and shows the values of the local variables in the Debug panel.
- **Flutter DevTools:** the widget inspector shows the tree and the size of every widget, and the performance view shows the frames that take longer than 16 milliseconds.
- **Error handling:** every asynchronous call is wrapped in try and catch, and setState is called only when mounted is true.
- **Database viewer:** the debug screen lists the rows of every table, which confirms that a form really saved its data.
- **Hot reload and hot restart:** a small change is applied with r and a full restart with R, without rebuilding the application.



Alerts Screen

19. Work Flow Of Screens

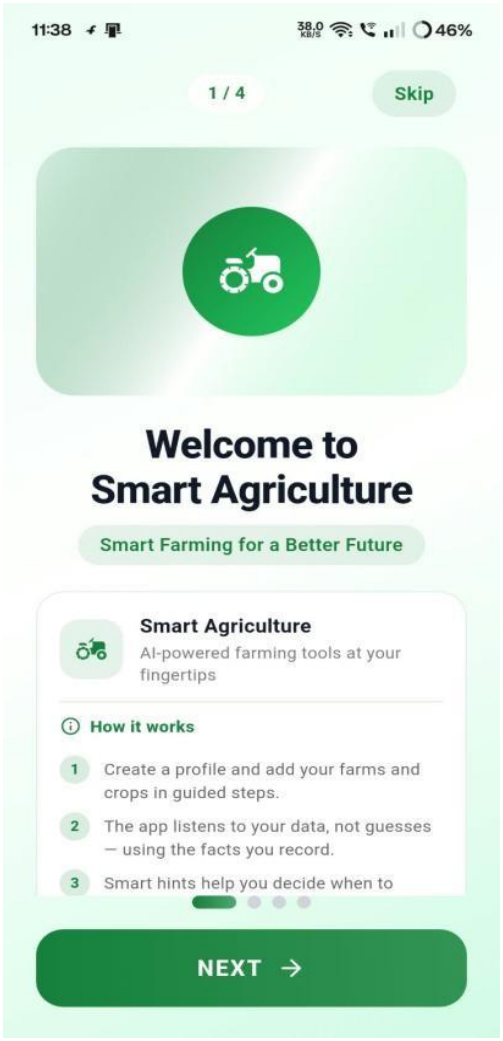
This section shows the complete workflow of the application, screen by screen, in the order in which a new user meets them. The first three screens introduce the application, the next two create the account, and the remaining screens are the working area of the farmer.

```
Splash Screen
  -> Onboarding 1 -> Onboarding 2 -> Onboarding 3 (Get Started)
       -> Login Screen -> Register Screen -> Dashboard
            |-- My Farms -> Add / Edit Farm -> Crop Monitoring
            |                                  -> Add / Edit Crop
            |-- Soil & Water -> Record Reading
            |-- Weather
            |-- Crop Scanner -> Scan Result -> Scan History
            |-- Ask Assistant -> Chat
            |-- Alerts
            `-- Profile -> Logout
```

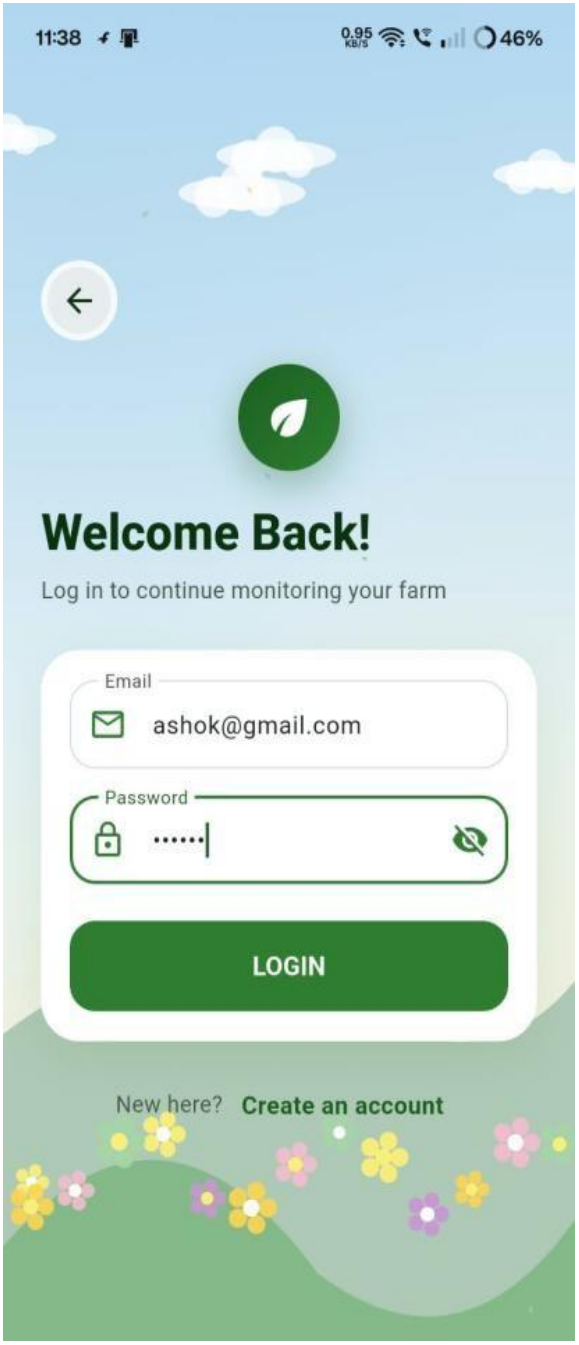
The screenshots below are taken from the running application on a 720 x 1600 Android device.



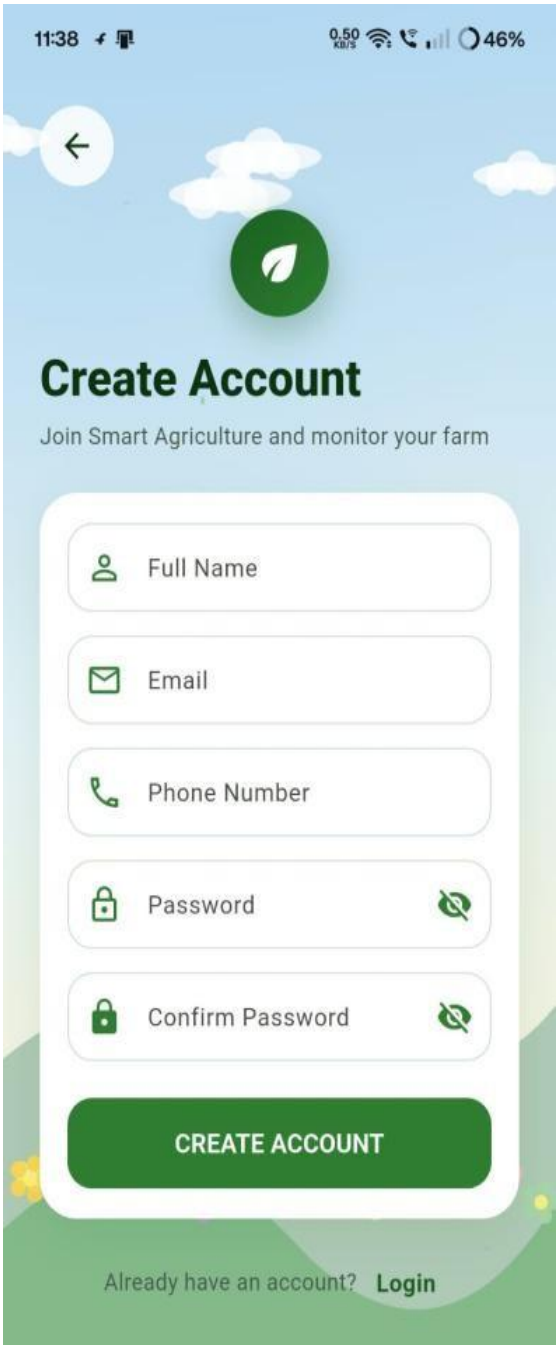
Splash Screen



Onboarding Screen



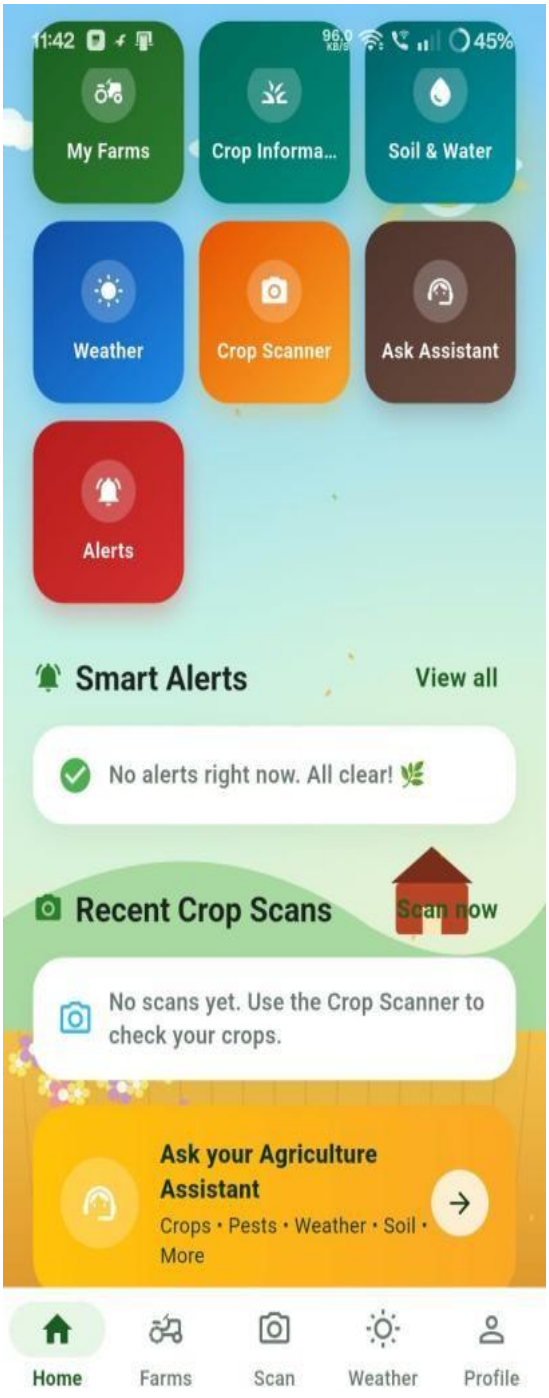
Login Screen



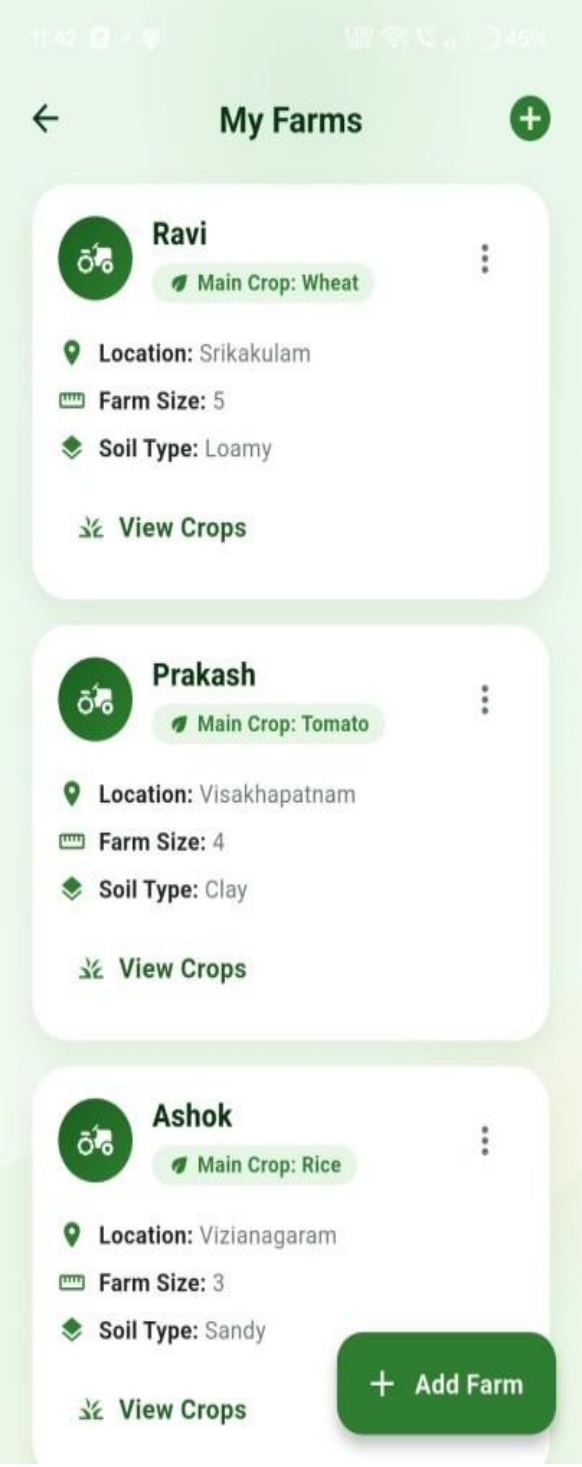
Registration Screen



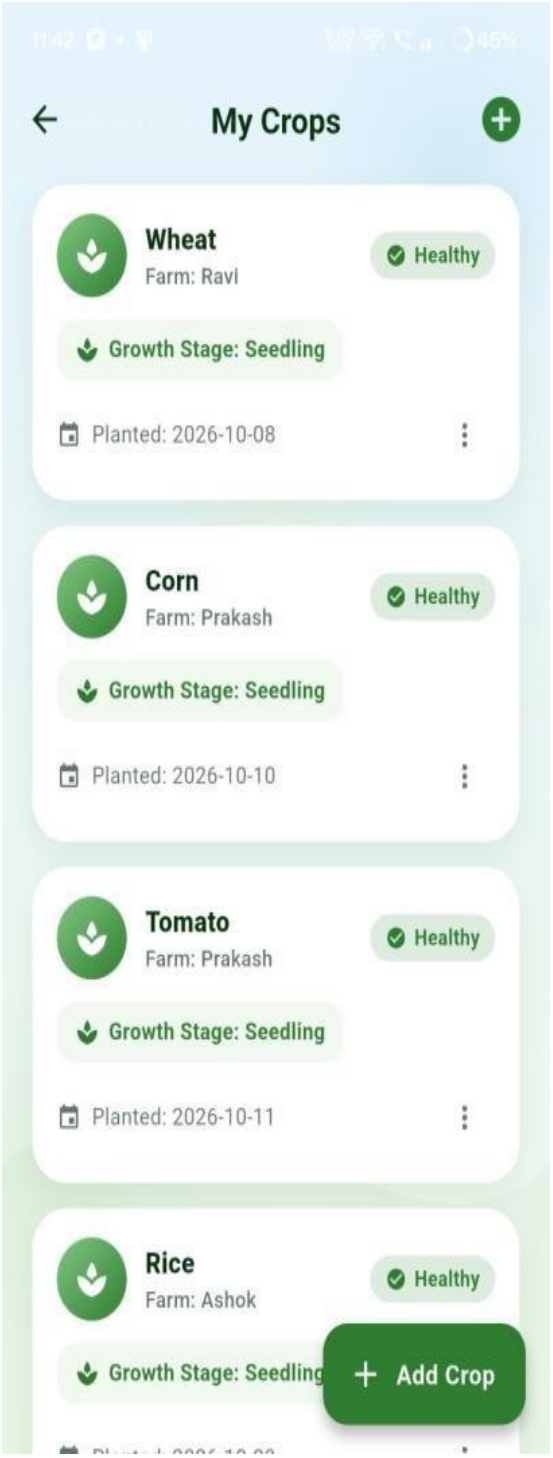
Dashboard Screen



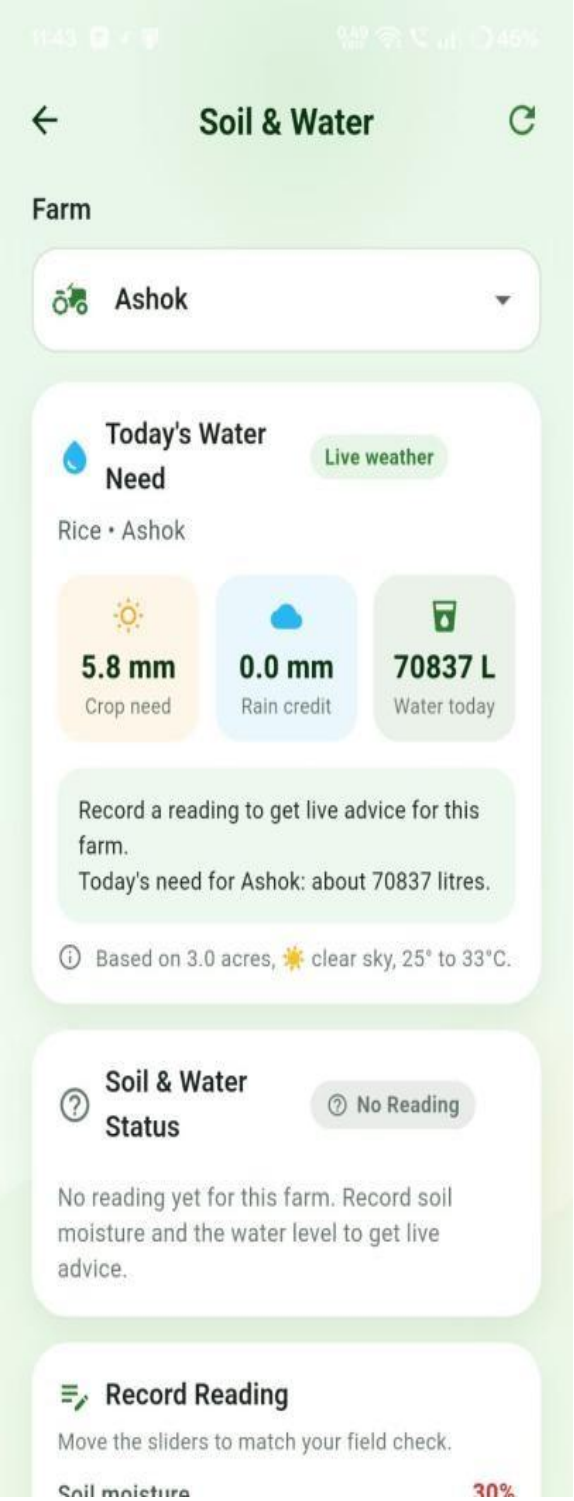
Dashboard Quick Actions



My Farms Screen



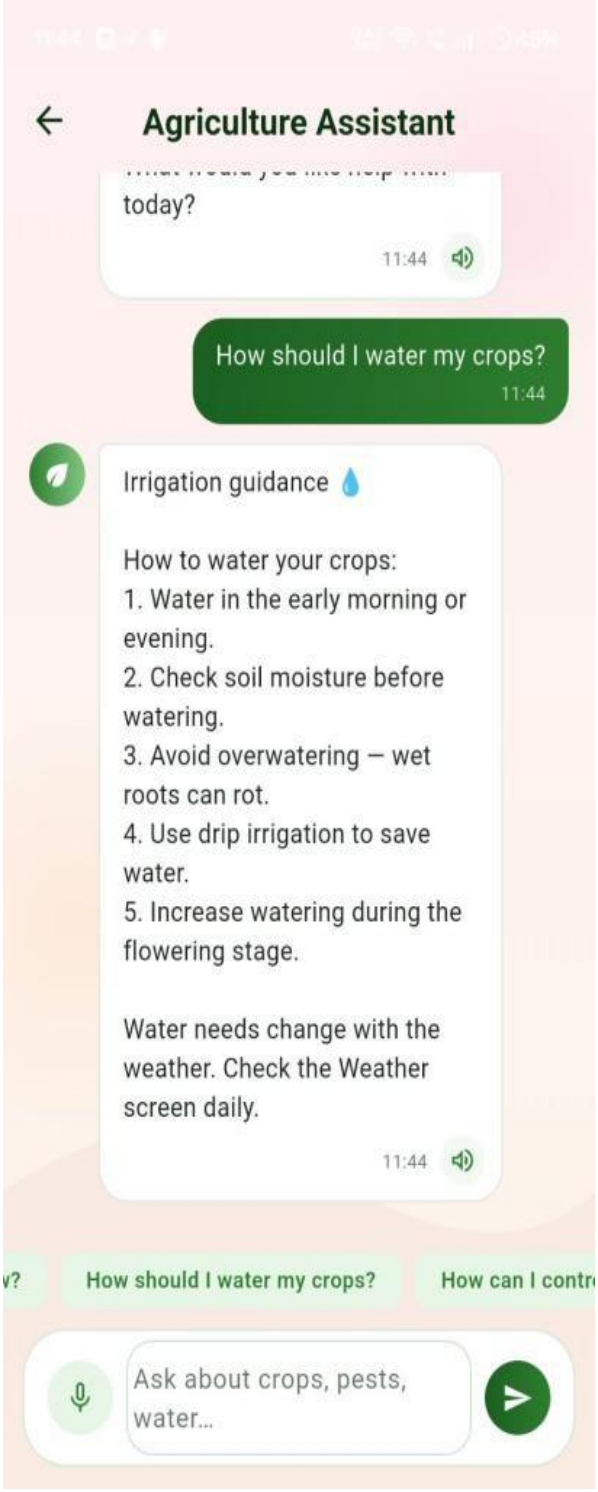
Crop Monitoring Screen



Soil Moisture Screen



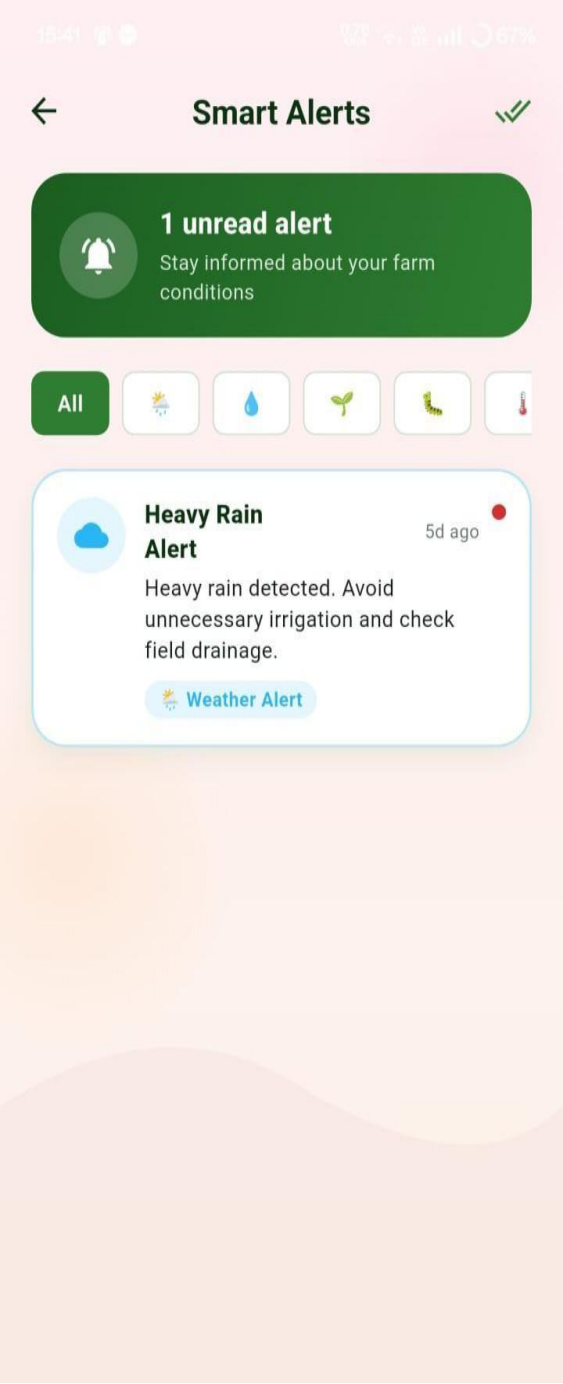
Weather Screen



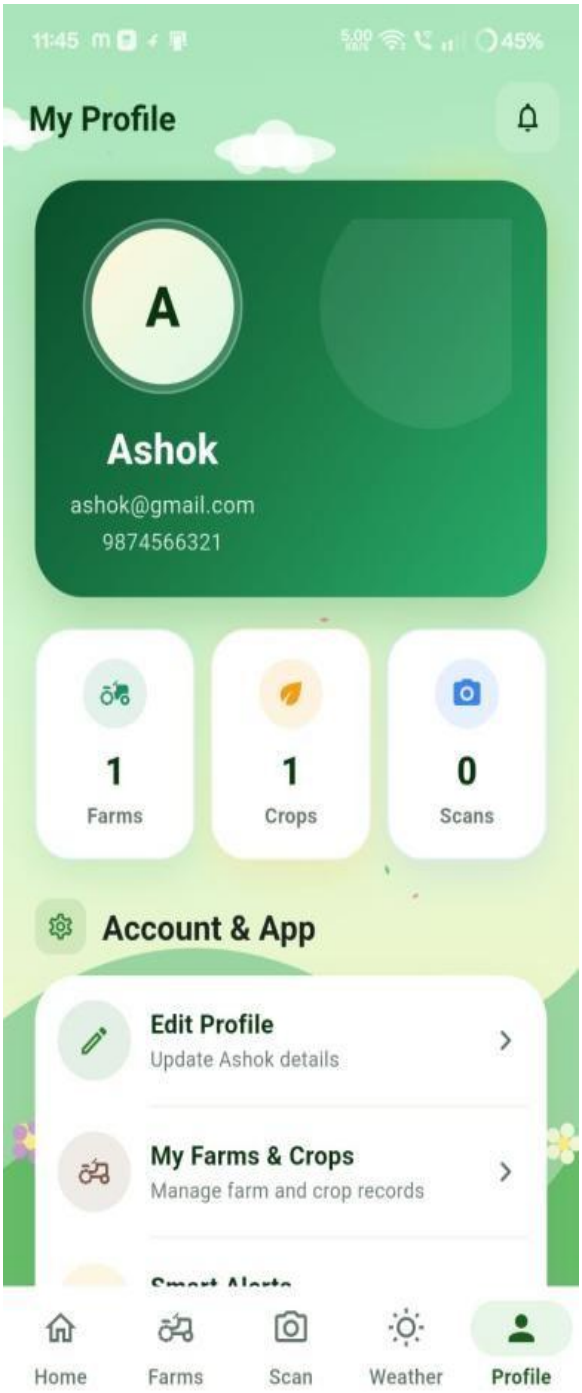
AI Assistant / Chat Screen



AI Crop Scanner Screen



Alerts Screen



Profile Screen

20. Conclusion

The **Smart Agriculture Monitoring App** was designed, implemented and tested as a complete Flutter application. It demonstrates the main concepts of the framework: a widget tree built from stateless and stateful widgets, navigation with routes and a bottom navigation bar, responsive layouts, forms with validation, custom painters, explicit animations, integration with a REST API and storage in a local SQLite database.

The application solves a practical problem. A farmer can keep the records of several farms and crops in one place, measure soil moisture and water level, see the daily water requirement in litres, follow the weather of the exact location of the farm, receive automatic alerts and ask an offline assistant for advice. Because the data is stored on the device, the application works in the field even when there is no network connection.

Learning Outcomes

- Understood the structure of a Flutter project and the role of the lib, assets, test and platform folders.
- Learned the Dart language, including variables, collections, classes, null safety and asynchronous programming with Future and async.
- Built reusable widgets and a consistent theme with a central colour palette and text styles.
- Used LayoutBuilder, MediaQuery and flexible widgets to support phone, tablet and desktop sizes.
- Implemented forms with validators, error messages and a loading state.
- Connected the application to a REST API with the http package and handled the JSON response and the network errors.
- Stored and queried relational data with SQLite and kept the SQL in a single helper class.
- Wrote unit and widget tests and used the analyzer, the debugger and DevTools to find and fix defects.

Future Enhancements

- Synchronise the local database with a cloud account so that the same records can be opened on another phone.
- Add a real machine learning model for the crop scanner instead of the rule-based analysis.
- Add charts for the moisture history of each farm and a monthly water usage report.
- Send push notifications when an alert is created and the application is not open.
- Connect soil moisture sensors through Bluetooth or an IoT gateway for automatic readings.

In conclusion, the project shows how a single Flutter codebase can deliver a useful, offline-first agricultural tool with a clean interface, reliable data storage and live weather information. The same structure can be extended with cloud sync and field sensors to reach more farmers.